

Cours Vue.js / Nuxt

Mission e-commerce La Poste

DT eCommerce — DSI BGPN

Préparation coding game et entretien

Livre format liseuse (texte noir, pages compactes)

À lire dans l'ordre, un chapitre à la fois

Comment lire ce livre

Ce livre rassemble tout le kit de préparation. Tu n'as pas besoin d'internet. Les exemples parlent d'une boutique La Poste (colis, timbres, cartons).

Lis d'abord les chapitres 1 à 6, puis le fil rouge du clic « Ajouter au panier ». Ensuite les chapitres 7 à 21. Les fiches de la deuxième partie se relisent la veille du test.

Sur une liseuse, agrandis un peu la police du lecteur si le code te paraît petit. Les blocs de code passent à la ligne pour rester lisibles.

Si un sigle bloque, va au glossaire (chapitre 20). Les questions du type « pourquoi .value ? » sont au chapitre 21.

Sommaire

1. Le projet La Poste
2. Comment une page web marche
3. Vue.js 3
4. Composition API
5. Nuxt, SSR et SEO
6. Pinia et Vuex
- Fil rouge — Un clic « Ajouter au panier »
7. TypeScript
8. Vite et Webpack
9. REST et GraphQL
10. Micro-services
11. SASS et BEM
12. Design system
13. Tests, TDD et BDD
14. Accessibilité WCAG
15. GitLab, Docker, CI/CD
16. ESLint et Prettier
17. Agile / Scrum
18. Performances et SEO

19. Visite guidée du mini-projet

20. Glossaire

21. Questions fréquentes

Stratégie du coding game

Cheatsheet Vue 3

Cheatsheet Nuxt et Pinia

Cheatsheet TypeScript, SASS, BEM, a11y

QCM d'entraînement

Entretien oral

Checklist jour J

Entraînement chrono

Snippets à recoller

Mapping fiche de poste

Première partie — Le cours

1. Le projet La Poste

De quoi on parle

Le **Groupe La Poste** n'est pas qu'un facteur. Il a une **Branche Grand Public et Numérique** (on dit **BGPN**) : tout ce que le client final voit sur internet (envoyer un colis, affranchir, suivre un recommandé, boutique).

La **DSI** (Direction des Systèmes d'Information) de cette branche construit et maintient ces sites. À l'intérieur, la **DT eCommerce** (Direction Technique e-commerce) s'occupe des **parcours d'achat et d'envoi** : catalogue, panier, paiement, suivi.

Toi, tu n'es pas embauché « pour faire du Vue ». Tu es là pour **améliorer l'expérience et la satisfaction client** sur ces parcours. La fiche le dit dès le contexte : c'est stratégique.

Pourquoi « moderniser le front »

Un site e-commerce vieux, c'est souvent :

- lent sur mobile (le client ferme) ;
- difficile à faire évoluer (un bouton casse trois pages) ;
- mauvais pour Google (le HTML arrive trop tard) ;
- inaccessible au clavier ou à un lecteur d'écran ;
- décalé par rapport au **design system** (chaque page a sa propre couleur de bouton).

La modernisation, dans cette offre, veut dire : **Vue 3 + Nuxt**, composants réutilisables, TypeScript, tests, accessibilité **WCAG 2.1**, performances, intégration API.

Ce que « prestation » veut dire

Tu intervies comme **consultant / freelance / ESN** dans une équipe déjà en place (souvent **Agile / Scrum**, parfois **SAFe** à l'échelle du groupe). Tu codes, tu revues, tu **encadres des juniors et des confirmés**. Ce n'est pas un stage d'exécution.

Les trois objectifs métier (à reciter)

1. **Des écrans performants et évolutifs** — le client charge vite, le code se maintient 3 ans.
2. **Une intégration fluide aux API** — REST et GraphQL déjà là ; tu ne réinventes pas le métier colis.
3. **SEO + temps de chargement + multi-navigateurs + accessibilité** — un retraité sur Safari, un client malvoyant, Google qui indexe « envoi colis ».

Si tu retiens une phrase :

Je construis le visage numérique des offres e-commerce La Poste, collé au design system, branché aux API, mesurable en satisfaction client.

Passe au chapitre 2 : sans HTML/CSS/JS, Vue n'a aucun sens.

2. Comment une page web marche

Imagine le site « envoyer un colis ». Trois langages se partagent le travail, **dans le navigateur** (Chrome, Firefox, Safari, Edge).

HTML — la structure

Le **HTML** (HyperText Markup Language) décrit **quoi** il y a sur la page, pas à quoi ça ressemble.

```
<h1>Envoi Colis S</h1>
<p>6,90 €</p>
<button type="button">Ajouter au panier</button>
```

- `<h1>` = le titre principal (un seul par page, important pour l'accessibilité et Google).
- `<p>` = un paragraphe.
- `<button>` = un vrai bouton. **Pas un `<div>` cliquable** : le clavier et le lecteur d'écran savent que c'est un bouton.

Le navigateur lit ce texte et construit un arbre en mémoire : le **DOM** (Document Object Model). Chaque balise = un nœud. JavaScript pourra **lire et modifier** cet arbre (changer le prix, ajouter une ligne au panier).

CSS — l'apparence

Le **CSS** (Cascading Style Sheets) dit **comment** ça s'affiche : couleurs, marges, grille.

```
button {  
  background: #1a1a1a;  
  color: white;  
}
```

Sans CSS, le site marche encore (on peut acheter), il est juste laid. Avec un mauvais CSS, le bouton « Commander » est illisible : là, **l'accessibilité** et le **design system** entrent en jeu (chapitres 11, 12, 14).

JavaScript — le comportement

Le **JavaScript** (souvent abrégé **JS**) réagit aux clics, parle au serveur, met à jour le DOM.

```
button.addEventListener("click", () => {  
  panier.push(produit);  
});
```

TypeScript (chapitre 7) n'est pas un autre langage magique : c'est du JavaScript **avec des types**, qui est ensuite **effacé** pour redevenir du JS que le navigateur comprend.

Le voyage d'une page (très important pour Nuxt)

Quand tu tapes une URL :

1. Le navigateur demande une page au **serveur**.
2. Le serveur répond, souvent avec du HTML.
3. Le navigateur affiche, puis télécharge CSS et JS.

4. Le JS s'exécute : la page devient interactive.

Deux grandes familles :

Site classique (plusieurs pages HTML)

Chaque clic « Catalogue » / « Panier » redemande une **nouvelle page** au serveur. Simple, bon pour le SEO, parfois lent (tout recharger).

SPA — Single Page Application

SPA = une seule page HTML, et le JavaScript **change le contenu** sans recharger. Vue sans Nuxt, c'est souvent ça. Sensation d'app fluide. **Problème** : au premier chargement, Google et le client voient parfois une page **vide** le temps que le JS s'exécute. Mauvais pour le **SEO** et le **LCP** (chapitre 18). C'est **la raison d'être de Nuxt / SSR**.

Front-end vs back-end vs API

- **Front-end** : ce qui tourne dans le navigateur (Vue, CSS). C'est **ton** métier ici.
- **Back-end** : ce qui tourne sur les serveurs La Poste (stock, prix, commande). Souvent d'autres équipes, **micro-services** (chapitre 10).
- **API** (Application Programming Interface) : le **contrat** entre les deux. Le front dit « donne-moi le produit lp-colis-s », le back répond en JSON.

```
{ "id": "lp-colis-s", "name": "Envoi Colis S",  
  "priceCents": 690 }
```

JSON (JavaScript Object Notation) = format texte d'objets, le plus courant pour les API web.

Framework : pourquoi Vue et pas « du JS brut »

En JS brut, pour une boutique tu réécrirais à la main : « si le panier change, mets à jour le badge, le total, le bouton ». C'est **verbeux et fragile**.

Un **framework** (Vue, React, Angular) te donne :

- des **composants** (une carte produit réutilisable) ;
- la **réactivité** (tu changes count, le badge se met à jour tout seul) ;
- une structure d'équipe (tout le monde code pareil).

Vue.js est le framework choisi par cette DSI. **Nuxt** est le **cadre** autour de Vue pour les sites avec routes, SEO, serveur.

Passe au chapitre 3.

3. Vue.js 3

L'idée en une phrase

Vue découpe l'écran en **composants**, et chaque composant a :

- un **template** (le HTML de ce morceau) ;
- un **script** (les données et les actions) ;
- éventuellement un **style** (le CSS de ce morceau).

Quand une **donnée réactive** change, Vue **re-rend tout seul** les endroits du template qui s'en servent. Tu ne fais plus `document.getElementById(...)`.

Un composant, concrètement

Fichier `ProductCard.vue` (c'est le nôtre dans le kit) :

```
<script setup lang="ts">
defineProps<{ product: Product }>();
</script>

<template>
  <h2>{{ product.name }}</h2>
  <p>{{ product.priceCents }}</p>
</template>
```

Les `{{ }}` s'appellent l'**interpolation** : Vue y met la valeur, et **surveille** `product.name`. Si le nom change, le `<h2>` change.

`lang="ts"` = ce script est du TypeScript.

`<script setup>` = la façon **moderne Vue 3** d'écrire un composant (Composition API, chapitre 4). Plus court que l'ancienne **Options API** (`data()`, `methods: { }`).

Pourquoi on découpe

Sans composants, une page « catalogue + filtres + panier » devient un fichier de 800 lignes. Avec des composants :

- `ProductFilters` = la barre recherche / catégorie / tri ;
- `ProductCard` = une carte ;
- `CatalogView` = la page qui assemble.

Chaque pièce a **une responsabilité**. En entretien : « je découpe par responsabilité, pas par taille de fichier ».

Le flux des données (règle d'or Vue)

Parent	--props-->	Enfant
Parent	<--emit---	Enfant

- Le parent **donne** des données à l'enfant via des **props** (propriétés). L'enfant **n'a pas le droit de les modifier**. C'est à sens unique, comme un formulaire papier qu'on te passe : tu le lis, tu ne le ratures pas.
- L'enfant **prévient** le parent via un **événement** (`emit`) : « le user a cliqué Ajouter ». Le parent décide alors (`cart.add(product)`).

Si tu modifies une prop dans l'enfant, Vue te le reproche, et tu auras un bug au coding game (exercice galerie d'images : on **copie** la liste dans un ref local, puis on splice la copie).

Les directives (attributs magiques de Vue)

Une **directive** commence par `v-`.

Directive	Rôle	Exemple boutique
<code>v-if / v-else-if / v-else</code>	Afficher ou détruire un bloc	« Chargement... » puis la liste
<code>v-show</code>	Cacher en CSS (display), le HTML reste	un panneau d'aide
<code>v-for</code>	Répéter	une <code></code> par produit
<code>v-model</code>	Lier un champ à une variable (aller-retour)	la recherche
<code>v-bind</code> ou <code>:</code>	Lier un attribut HTML à du JS	<code>:src</code> , <code>:disabled</code>
<code>v-on</code> ou <code>@</code>	Écouter un événement	<code>@click="add"</code>

Exemple qui combine :

```
<button type="button" :disabled="product.stock === 0"
@click="emit('add', product)">
  Ajouter au panier
</button>
```

- `:disabled="..."` = si stock 0, le bouton est grisé (HTML natif, donc clavier-friendly).
- `@click` = au clic, on **émet** vers le parent.

v-if vs v-show (question QCM)

- `v-if="false"` : le bloc **n'existe pas** dans le DOM. Coût à chaque fois qu'on le recrée.
- `v-show="false"` : le bloc existe, mais `display: none`. Plus cheap si on toggle souvent.

Pour un chargement de catalogue, `v-if` est naturel (on n'a pas encore les données). Pour un paragraphe « afficher / masquer l'aide », `v-show` suffit (exercice TestDome classique).

v-for et la key (question QCM, bug fréquent)

```
<li v-for="product in filtered" :key="product.id">
```

La **key** est l'identité de la ligne. Vue s'en sert pour **réutiliser** le bon DOM. Si tu mets `:key="index"` (0, 1, 2...) et que tu filtres ou supprimes, Vue recycle la mauvaise carte : un champ quantité « saute » d'un produit à l'autre.

Toujours un id métier stable, jamais l'index, dès que la liste peut changer.

Vue 2 vs Vue 3 (l'offre dit 3.x)

L'offre demande **Vue.js 3.x**. Différences utiles à l'oral :

	Vue 2	Vue 3
API recommandée	Options API	Composition API + <code><script setup></code>
Réactivité tableaux	pièges (<code>Vue.set</code>)	Proxy : push marche

	Vue 2	Vue 3
State	Vuex	Pinia (Vuex existe encore en legacy)
Bundler usuel	Webpack / Vue CLI	Vite
Fragments	un seul élément racine	plusieurs racines OK

Si le coding game te donne un CDN Vue 2, tu retombes sur Options API. Sinon, tu codes en Vue 3.

Ce que Vue ne fait pas tout seul

- **Les routes** (/panier, /commande) → Vue Router, ou **Nuxt** qui le fait pour toi.
- **Le SEO / HTML serveur** → **Nuxt SSR**.
- **Le panier partagé entre 6 pages** → **Pinia**.
- **Parler au back** → fetch / \$fetch / Apollo GraphQL.

Vue = le moteur de l'UI. Le reste, ce sont les outils de l'offre.

Chapitre 4 : on ouvre le capot de la Composition API.

4. Composition API

La **Composition API**, c'est la façon Vue 3 d'organiser la logique **par fonction**, dans `<script setup>`, au lieu de cases `data / methods / computed` de l'Options API.

Pourquoi ils l'aiment : tu regroupes « tout ce qui concerne le panier » au même endroit, tu **réutilises** cette logique dans un **composable** (`useProducts`, `useCart`), et TypeScript s'y retrouve.

ref : une boîte dont Vue surveille le contenu

```
import { ref } from "vue";

const count = ref(0);
count.value += 1; // en script, TOUJOURS .value
```

Dans le **template**, Vue **déballe** tout seul : tu écris `{{ count }}`, pas `{{ count.value }}`.

Analogie : `ref` est une **boîte**. Vue surveille la boîte. Si tu sors l'objet de la boîte (`const n = count` puis tu n'utilises plus `count`), Vue ne voit plus les changements. D'où le piège **destructuration** avec Pinia (chapitre 6).

`ref` marche pour : un nombre, une string, un booléen, un tableau, un objet.

reactive : un objet déjà « magique »

```
import { reactive } from "vue";

const form = reactive({ email: "", name: "" });
form.email = "a@b.c"; // pas de .value
```

Règle simple le jour J :

- primitive (0, "", true) → **ref** ;
- formulaire à 3 champs → **reactive** ou un `ref({ ... })` ;
- **en cas de doute, ref partout.**

Piège : `const { email } = form` **casse** la réactivité (tu as copié la string, plus l'objet magique). Utiliser `toRefs(form)` si tu dois déstructurer.

computed : une valeur dérivée, toujours à jour

```
const totalCents = computed(() => qty.value *
priceCents.value);
```

Tu **ne stockes pas** le total à la main. Tu le **décis**. Vue le recalculé quand `qty` ou `priceCents` change, et **seulement** à ce moment (cache).

Règle : **pas d'effet de bord** dans un `computed` (pas de `fetch`, pas de `console.log` métier, pas de `array.push`). Un `computed` est une **formule**, comme dans Excel.

C'est **la** bonne place pour : filtre de catalogue, total panier, isValid d'un formulaire, libellé « 2 articles ».

watch et watchEffect : réagir à un changement

```
watch(query, (next, prev) => {  
  // lancer une recherche API, avec debounce  
});
```

- **watch** : tu **choisis** la source. Ne s'exécute **pas** tout de suite (sauf { immediate: true }).
- **watchEffect** : Vue **devine** les dépendances en exécutant la fonction **immédiatement**, puis à chaque fois qu'une dépendance change.

Pour un filtre **local** (tableau déjà en mémoire) : **computed**, pas watch. Pour appeler une API quand la recherche change : **watch** + debounce.

Les props

```
const props = defineProps<{  
  product: Product;  
  compact?: boolean; // optionnel  

```

? = le parent peut ne pas l'envoyer. En template tu utilises product directement (unwrap). En script : props.product.

Les props sont **en lecture seule**.

Les événements (emit)

```
const emit = defineEmits<{
  add: [product: Product];
}>();

emit("add", props.product);
```

Le parent écrit `@add="cart.add"` (ou `@add="onAdd"`). C'est le **contrat** : l'enfant dit « add », il n'importe pas le store s'il peut l'éviter. Un composant **bête** (carte) est plus testable et plus Storybook-friendly.

v-model sur un composant

v-model sur un `<input>` natif = la valeur **et** les frappes.

Sur **ton** composant, ça veut dire par convention :

- le parent passe `modelValue` ;
- l'enfant émet `update:modelValue`.

Vue 3.4+ : `const model = defineModel<string>()`. Dans le kit, `ProductFilters` utilise `v-model:query` (un **v-model nommé**) : le parent écrit `v-model:query="query"`, l'enfant émet `update:query`.

Cycle de vie

Un composant **naît**, **s'affiche**, **se met à jour**, **meurt**.

Hook	Quand	Usage typique
onMounted	le HTML du composant est dans la page	fetch en SPA, mesurer un élément
onUnmounted	le composant disparaît	clearTimeout, enlever un listener
onUpdated	après un re-render	rare ; souvent un watch suffit

En **Nuxt SSR**, onMounted **ne tourne que dans le navigateur**, pas sur le serveur. D'où : ne pas mettre le fetch SEO uniquement dans onMounted (chapitre 5).

Un composable : useQuelqueChose

Un **composable** est une fonction qui **utilise** ref / computed et les **retourne**. Convention : préfixe use.

```
export function useProducts() {
  const query = ref("");
  const filtered = computed(() => /* ... */);
  return { query, filtered };
}
```

Dans deux pages, tu réutilises la même logique. C'est **mieux** qu'un mixin Vue 2 (illisibles, collisions de noms).

Dans le kit : src/composables/useProducts.ts charge le catalogue, filtre, trie.

Options API en 30 secondes (si le test est vieux)

```
export default {
  props: ["links"],
  data() {
    return { images: [...this.links] };
  },
  computed: {
    count() { return this.images.length; },
  },
  methods: {
    remove(i) { this.images.splice(i, 1); },
  },
  mounted() {},
};
```

Équivalences : data $\approx$ ref, computed = computed, methods = function, mounted = onMounted.

Chapitre 5 : Nuxt pose ce moteur Vue **sur un serveur**, pour le SEO.

5. Nuxt, SSR et SEO

Le problème que Nuxt résout

Avec une **SPA Vue** pure :

1. le serveur envoie une page HTML **presque vide** (<div id="app"></div>);
2. le navigateur télécharge un gros fichier JS ;
3. Vue se réveille et **construit** le catalogue ;
4. **seulement là**, l'utilisateur et Google voient « Envoi Colis S ».

Sur un site **grand public** (La Poste), c'est grave :

- le client 4G voit un blanc (mauvais **LCP**, chapitre 18) ;
- Google indexe mal (mauvais **SEO** — Search Engine Optimization, le fait d'être trouvable sur Google).

Nuxt est un **méta-framework** : Vue + un serveur + des conventions de dossiers. Il peut **rendre le HTML du catalogue sur le serveur** et l'envoyer **déjà rempli**. Ensuite le navigateur « hydrates » : il attache les clics (ajouter au panier) sans tout reconstruire.

Quatre modes, en français

Nom	Ce qui se passe	Exemple La Poste
CSR (Client-Side Rendering)	tout est peint par le JS dans le navigateur	espace connecté, étape paiement

Nom	Ce qui se passe	Exemple La Poste
SSR (Server-Side Rendering)	le serveur fabrique le HTML à chaque requête , puis hydrate	fiche produit, home
SSG (Static Site Generation)	le HTML est fabriqué au build , fichiers statiques	mentions légales
ISR / SWR	HTML en cache, régénéré de temps en temps	catalogue qui bouge peu

Phrase d’entretien :

Catalogue et fiches en SSR (SEO + LCP). Tunnel de commande souvent plus CSR, car privé, pas indexé, très interactif.

L’arborescence (convention = moins de config)

pages/index.vue	→ URL /
pages/panier.vue	→ URL /panier
pages/produits/[id].vue	→ URL /produits/lp-colis-s
layouts/default.vue	→ le chrome (header, footer)
components/ProductCard.vue	→ auto-importé, pas besoin d’import
composables/useCart.ts	→ auto-importé
stores/cart.ts	→ Pinia
middleware/auth.ts	→ garde avant d’entrer dans une page
server/api/products.get.ts	→ URL /api/products côté serveur Nuxt
nuxt.config.ts	→ la config

Les **crochets** [id] = paramètre dynamique. Dans la page :

```
const route = useRoute();  
const id = route.params.id; // "lp-colis-s"
```

Tu n'installes pas Vue Router à la main : Nuxt le câble.

useFetch vs useAsyncData vs \$fetch

C'est la question QCM Nuxt.

useFetch("/api/products")

À utiliser **dans le setup d'une page / composant**, pour un GET simple. Nuxt :

- lance l'appel **déjà sur le serveur** (SSR) ;
- envoie le résultat dans la page (**payload**) ;
- **ne refait pas** le même appel au moment de l'hydratation (pas de double fetch).

```
const { data, pending, error, refresh } = await  
useFetch("/api/products");
```

- data = les produits (ref) ;
- pending = chargement ;
- error = échec réseau / HTTP ;
- refresh = relancer.

useAsyncData("clé", () => ...)

Quand tu dois **combiner** deux appels, transformer, ou donner une **clé de cache** unique (product-`{id}`).

\$fetch

Le couteau suisse, **partout** : dans un clic, dans un store, dans un handler.

```
async function appliquerCode(code: string) {
  await $fetch("/api/coupon", { method: "POST", body: {
    code } });
}
```

Interdit : useFetch à l'intérieur d'un @click. useFetch est un composable de **setup**, pas une fonction d'événement.

Hydration mismatch (piège)

Le HTML du serveur doit **matcher** le premier rendu navigateur. Sinon Vue râle.

Causes classiques :

- `Date.now()`, `Math.random()` pendant le render ;
- lire `window` ou `localStorage` pendant le render SSR (ça n'existe pas sur le serveur) ;
- un `v-if` qui dépend du fuseau horaire.

Correct : `import.meta.client / <ClientOnly>` pour le morceau « widget carte », `localStorage` dans `onMounted`.

SEO avec Nuxt

```
useSeoMeta({
  title: "Envoi Colis S – La Poste",
  description: "Colis jusqu'à 2 kg, suivi inclus.",
  ogTitle: "Envoi Colis S",
  ogImage: "https://.../colis-s.jpg",
});
```

- **title / description** : onglet + Google.
- **og:*** : Open Graph, l'aperçu quand on partage sur WhatsApp / LinkedIn.

html lang="fr" : obligatoire, accessibilité + SEO France.

routeRules (idée à citer)

Dans `nuxt.config.ts` tu peux dire : home en cache 60 s, tunnel sans SSR.

```
routeRules: {
  "/": { swr: 60 },
  "/commande/**": { ssr: false },
}
```

Tu n'as pas à le recoder au coding game. Tu dois **savoir que ça existe**.

Nuxt 2 vs Nuxt 3

L'offre dit « Nuxt.js » sans version. En 2026, une modernisation BGNP = **Nuxt 3** (Vue 3, Vite, Pinia). Du **Nuxt 2** (Vue 2, Webpack, Vuex) peut rester en legacy. À l'oral : « je suis Nuxt 3, je sais lire du Nuxt 2 ».

Notre mini-projet du kit est une **SPA Vite** (plus simple à lancer pour s'entraîner). Les **idées** SSR, tu les as dans ce chapitre et les QCM. Si le coding game est Nuxt, remplace `onMounted` + `fetch` par `useFetch`.

Chapitre 6 : où vit le panier.

6. Pinia et Vuex

Le problème

Tu as un badge « Panier (2) » dans le **header**, une liste dans la page **catalogue**, un total dans la page **panier**. Ces trois endroits doivent **voir la même vérité**.

Si chaque composant a son propre `ref()`, ils **ne se parlent pas**.

Solutions, de la plus petite à la plus large :

1. **Props / emit** — OK pour un parent et un enfant (carte → catalogue).
2. **Provide / inject** — OK pour un thème, une locale, pas pour un panier métier.
3. **Store Pinia** — la solution Vue 3 pour un état partagé : panier, user connecté, préférences.

Analogie : le store est le **cahier de caisse** du magasin. Personne ne compte dans sa tête dans un rayon différent.

Pinia (à maîtriser)

Pinia est le gestionnaire d'état **officiel** de Vue 3. Un store = une fonction `defineStore`.

Deux styles existent. On utilise le **style setup** (identique à un composable) :

```
export const useCartStore = defineStore("cart", () => {  
  const items = ref<CartItem[]>([]);           // state  
  const count = computed(() => /* ... */);      // getter  
  function add(product: Product) { /* ... */ } // action  
  return { items, count, add };  
});
```

Dans un composant :

```
const cart = useCartStore();  
const { count, items } = storeToRefs(cart); //  
indispensable pour garder la réactivité  
cart.add(product);                          // les  
fonctions : PAS dans storeToRefs
```

Le piège n°1 de tout QCM Pinia / Vuex

```
const { count } = useCartStore(); // count n'est PLUS  
réactif
```

Tu as **sorti** le nombre de la boîte. Le header ne bougera plus.

Correct : storeToRefs.

Règles d'un bon store panier

- Une **seule** source de vérité (items).
- count et total sont des **computed**, pas des variables que tu incréments à la main (tu les oublierais).
- add refuse le stock 0 (règle métier dans le store, pas dans trois boutons).

- On ne met **pas** le HTML dans le store. Le store ne connaît pas les couleurs.

Dans le kit : `src/stores/cart.ts` + tests `tests/cart-store.test.ts`.

Vuex (l'offre écrit « Vuex » / « Vueex »)

Vuex est l'ancien store, standard de Vue 2. Tu dois pouvoir le **mapper** en 20 secondes, même si tu codes Pinia.

Vuex	Rôle	Équivalent Pinia
state	les données	ref / reactive
getters	valeurs dérivées	computed
mutations	changements synchrones , les seules autorisées à écrire le state	<i>n'existe plus</i> : tu mutes dans l'action
actions	async (appel API), elles commit une mutation	fonctions du store
modules	découper (cart, user)	plusieurs stores (useCartStore, useUserStore)

Pourquoi Pinia a gagné : plus simple, meilleur TypeScript, plus besoin de mutations, Devtools propres.

Phrase orale :

Sur du legacy Vue 2, je lis Vuex. Sur du Vue 3 / Nuxt 3, je pose Pinia, un store par domaine, et je migre module par module si

besoin.

Où ne PAS mettre l'état

- Le résultat d'un **filtre de recherche** : souvent local à la page (`useProducts`), ce n'est pas global.
- Un `v-model` de champ : local.
- Le **catalogue entier** : parfois cache dans un store, souvent `useFetch` de page (surtout en SSR).

Question à se poser : « est-ce que **deux pages** ont besoin de la même donnée en même temps ? » Oui → store. Non → composable / ref local.

Persister le panier (oral, rarement le test)

`localStorage` survit au F5. Attention au SSR : `localStorage` n'existe pas sur le serveur. Lire au `onMounted` ou `import.meta.client`. Plugin fréquent : `pinia-plugin-persistedstate`.

À La Poste, le vrai panier est souvent **côté serveur** (compte, multi-device). Le front affiche, le POST commande est la vérité (stock 409).

Chapitre 7 : donner un contrat à ces objets avec TypeScript.

Fil rouge — Un clic « Ajouter au panier »

Relis ça **après** les chapitres 2 à 6. C'est la même histoire vue par toutes les couches. Si tu sais la raconter, tu sais l'oral **et** le coding game.

1. Le designer (Figma / Frontify)

La carte a un bouton primaire noir, un état disabled, un prix en gras. Les tokens viennent du DS. Toi tu ne choisis pas un bleu « plus moderne ».

2. Le HTML (composant ProductCard)

Un vrai `<button type="button">`. Label accessible. Classe BEM `product-card__cta`. `:disabled` si stock 0.

3. Vue (réactivité)

Le clic fait `emit('add', product)`. Pas de `document.querySelector`. Le parent écoute `@add`.

4. Le parent (CatalogView)

Il connaît le **métier de la page** : `cart.add(product)`. Il ne calcule pas le CSS.

5. Pinia (useCartStore)

items est la vérité. add refuse la rupture, incrémente si le SKU existe déjà. count et totalCents sont des **computed** : le badge du header **bouge tout seul**.

6. TypeScript

product: Product : tu ne peux pas add({ nom: "colis" }) par erreur. priceCents est un number.

7. L'API (en vrai, pas dans le kit)

Souvent POST /api/cart/items. Réponse 200 ou **409 stock**. Le store se **resynchronise**. Ici on simule en mémoire pour le test chrono.

8. Micro-services

Le BFF parle au service stock + panier. Le navigateur n'appelle pas trois hosts.

9. Accessibilité

Le badge aria-live annonce « 2 ». Le focus reste sur le bouton. On n'a pas sauté de page sans prévenir.

10. Test

Un test store : « deux add du même produit → quantity 2 ». Un futur Cypress : le badge se voit. TDD = on a d'abord écrit cette phrase, ensuite le if (existing).

11. CI

Push → GitLab npm test → vert → revue → Docker recette.

12. Perf / SEO

Ce clic n'a **pas** besoin de SSR. En revanche la **carte visible** au premier affichage du catalogue, oui : HTML serveur Nuxt, image pas trop lourde, pas de CLS.

Quand tu codes le jour J, tu rejoyes mentalement les étapes **3-5-6-9**. Le reste, tu l'**orales**.

7. TypeScript

Pourquoi l'offre exige TypeScript

JavaScript est **permissif** : tu peux faire `product.prix` alors que le champ s'appelle `priceCents`. Ça plante **chez le client**, pas dans l'éditeur.

TypeScript (TS) est du JavaScript + un **filet**. Tu décris la forme des données. L'éditeur hurle **pendant que tu codes**. Au build, `tsc / vue-tsc` **efface** les types : le navigateur reçoit du JS normal.

L'offre dit « maîtrise de TypeScript ». En coding game, ça veut dire : **interfaces**, pas de `any`, props typées.

Les types dont tu as besoin

Types primitifs

`string`, `number`, `boolean`, `null`, `undefined`.

En e-commerce : un prix est un **number en centimes** ($690 = 6,90 \text{ €}$), pas un `"6,90"` (les strings ne s'additionnent pas comme tu crois : `"1" + "2" = "12"`).

interface : le contrat d'un objet

```
interface Product {  
  id: string;  
  name: string;  
  priceCents: number;  
}
```

```
    stock: number;  
}
```

Tout objet `Product` **doit** avoir ces champs. Si l'API oublie `stock`, TypeScript te prévient **si** tu as typé la réponse.

type union : un choix fermé

```
type ProductCategory = "colis" | "affranchissement" |  
"emballage" | "boutique";
```

Impossible d'écrire `"Colis"` par erreur (majuscule). Le select de filtre reste aligné.

interface vs type : pour un objet métier, interface est idiomatique. Pour une union ou un alias, type. Les deux passent au test.

Tableaux et fonctions

```
function add(product: Product, qty: number = 1): boolean {  
    if (product.stock < qty) return false;  
    return true;  
}  
  
const list: Product[] = [];
```

: `boolean` après `)` = type de **retour**. Ça documente : `add` ne renvoie pas le panier, il dit succès / refus.

Optionnel et Pick

```
compact?: boolean; // peut manquer
Pick<Product, "id" | "name">; // un sous-ensemble
```

any vs unknown (QCM)

- **any** : « ferme le filet ». Plus aucune vérif. **Interdit** sauf debug de 30 secondes.
- **unknown** : « je ne sais pas encore ». Tu **dois** tester avant d'utiliser (typeof, in, type guard).

```
function isProduct(x: unknown): x is Product {
    return typeof x === "object" && x !== null && "id" in x
    && "priceCents" in x;
}
```

Utile après un `JSON.parse` (qui renvoie `any` en pratique : enveloppe-le).

Génériques, version simple

Une fonction qui marche **pour plusieurs types**, sans tout casser :

```
function uniqueBy<T, K>(items: T[], keyFn: (item: T) => K):
T[] {
    // T = le type des éléments, K = le type de la clé
}
```

Tu n'as pas besoin d'écrire des génériques complexes au coding game. Comprendre `<T>` = « à remplir plus tard » suffit.

Dans Vue

```
defineProps<{ product: Product }>();  
const items = ref<CartItem[]>([]);
```

`ref<CartItem[]>([])` : sans le `<CartItem[]>`, TypeScript croit que c'est `never[]` et refuse `push`.

Ce que TypeScript ne fait pas

Il **n'empêche pas** une API de renvoyer un stock faux. Il empêche **toi** de te tromper de champ. La validation métier (email, adresse) reste **ton** code (voir checkout du kit).

Chapitre 8 : comment ce TS devient un site dans le navigateur.

8. Vite et Webpack

Le problème

Le navigateur ne comprend pas nativement, tel quel, un projet de 80 fichiers `.vue`, du TypeScript, du SASS, des `import`. Il faut **transformer** et **emballer**.

L'outil qui fait ça s'appelle un **bundler** (emballeur) :

- en **dev** : serveur local, rechargement rapide (`npm run dev`) ;
- en **prod** : fichiers minifiés, découpés, hashés (`npm run build` → dossier `dist/` ou `.output/`).

L'offre dit **Webpack** ou **Vite**. Les deux font le même métier, à des époques différentes.

Webpack

Webpack a dominé 2016–2021 (Vue CLI, **Nuxt 2**). Il **assemble tout** même en dev : plus lent à démarrer sur un gros projet. Config souvent lourde (`webpack.config.js`).

Tu dois savoir : loader (un fichier `.vue` passe dans `vue-loader`, le `.scss` dans `sass-loader`), plugin, code splitting (couper le JS par page pour ne pas tout télécharger d'un coup).

Vite (prononcer « vite », c'est voulu)

Vite est le défaut de **Vue 3** et **Nuxt 3**. En dev, il s'appuie sur les **modules ESM natifs** du navigateur : il ne bundle pas tout au démarrage, il transforme **à la demande**. D'où le « instantané ».

En production, Vite utilise **Rollup** (un autre emballeur, très bon pour les libs et le tree-shaking : enlever le code mort).

Fichier `vite.config.ts` du kit : `plugin vue()`, alias `@` → dossier `src`. Écrire `import x from "@stores/cart"` au lieu de `../../../../../`.

Variables d'environnement

Dans Vite, les variables exposées au front doivent commencer par `VITE_` :

```
import.meta.env.VITE_API_URL
```

Jamais un secret (clé privée, mot de passe) dans `VITE_*` : tout le JS part chez le client, n'importe qui le lit. Les secrets restent au **BFF / serveur**.

`process.env` est un réflexe Node / Webpack. Dans un coding game Vite, c'est `import.meta.env`.

Ce que tu fais concrètement le jour J

Tu ne **configures** presque jamais Webpack. Tu dois :

- savoir **pourquoi** ça compile ;
- connaître `npm run dev / build / preview` ;
- dire « Nuxt 3 = Vite, Nuxt 2 = Webpack » ;
- ne pas installer Webpack dans un projet Vite « par habitude ».

Chapitre 9 : une fois le front lancé, il parle au back.

9. REST et GraphQL

Le front **n'a pas** la base produits. Il **demande**. L'offre exige REST et GraphQL.

REST, l'idée

REST (Representational State Transfer) = on manipule des **ressources** via des **URL** et des **verbes HTTP**.

Verbe	Intention	Exemple
GET	lire, sans modifier	GET /api/products
POST	créer	POST /api/cart/items { sku, qty }
PUT / PATCH	remplacer / modifier	changer la quantité
DELETE	supprimer	retirer une ligne

Le serveur répond avec un **code HTTP** :

Code	Sens	Ce que le front affiche
200	OK	la liste
201	créé	ligne ajoutée
400	requête mal formée	« données invalides »
401	pas connecté	rediriger login
403	connecté mais pas le droit	message métier
404	ressource inconnue	page produit introuvable
409	conflit (souvent stock)	« plus assez en stock »
422	validation métier	erreurs de champs

Code	Sens	Ce que le front affiche
500	le serveur a cassé	« service indisponible », log Sentry

Tu n'affiches **jamais** la stack trace. Tu affiches un français clair.

En JS :

```
const res = await fetch("/api/products");
if (!res.ok) throw new Error("HTTP " + res.status);
const products: Product[] = await res.json();
```

En Nuxt : useFetch / \$fetch font le JSON pour toi et lèvent sur erreur.

JSON

Le corps est presque toujours du **JSON** : { "id": "...", "priceCents": 690 }. JSON.parse peut **** exploser**** si le body n'est pas du JSON → try/catch (QCM).

GraphQL, l'idée

GraphQL = une **seule URL** (souvent /graphql) et tu **décris** exactement les champs voulus.

```
query Products($q: String) {
  products(search: $q) {
    id
    name
    priceCents
    stock
  }
}
```

```
}  
}
```

Avantage : sur une fiche produit riche (avis, reco, dispo bureau), tu évites le **over-fetch** (télécharger 40 champs pour en afficher 4). Tu évites aussi 6 GET REST différents si le serveur est bien fait (**under-fetch**).

Inconvénients à citer :

- les erreurs peuvent être **partielles** : data **et** errors[] en même temps ;
- le cache est plus subtil qu'un GET REST ;
- sans BFF, le front peut déclencher un **N+1** (une requête par carte).

Tu n'as pas besoin d'écrire un serveur GraphQL au coding game. Tu dois **lire** une query et savoir useQuery / \$fetch('/graphql').

REST vs GraphQL : phrase unique

*REST : une URL par ressource, simple, cache HTTP naturel.
GraphQL : une URL, le front choisit les champs, idéal fiches riches. À La Poste, les deux coexistent ; le front parle à un BFF qui agrège.*

CORS, cookies, secrets (oral 30 s)

- **CORS** : le navigateur bloque un site `a.fr` qui appelle `api.b.fr` si le serveur n'autorise pas. Le **BFF** (même domaine) évite une partie du problème.
- Session : cookies **HttpOnly** (le JS ne les lit pas → moins de vol XSS).
- **XSS** : ne jamais injecter du HTML API avec `v-html` sans filtre.
- Pas de token secret dans le front.

Chapitre 10 : qui se cache derrière ces URL.

10. Micro-services

Monolithe vs micro-services

Un **monolithe** : une seule grosse application back qui fait catalogue + stock + paiement + suivi. Simple au début, dur à faire évoluer à l'échelle d'un groupe.

Une architecture **micro-services** : **plusieurs petits programmes**, chacun **un métier**, déployés **indépendamment**.

Exemples plausibles e-commerce postal :

- service **catalogue** (nom, prix affiché, visuels) ;
- service **stock / éligibilité** ;
- service **panier / commande** ;
- service **paiement** ;
- service **tracking colis** ;
- service **compte client**.

L'offre dit « maîtrise d'une architecture micro-service ». On ne te demande pas de coder Kubernetes. On te demande de **ne pas concevoir le front comme s'il y avait une seule base magique**.

Le front ne parle pas à 8 services

Si chaque carte produit appelle 4 hosts différents :

- cauchemar CORS, auth, versions, timeouts ;

- 4 pannes possibles par page ;
- tu couples l'UI à l'orga interne.

La solution s'appelle **BFF** (Backend For Frontend) ou **API Gateway** :

Navigateur → Nuxt / BFF → micro-services

Un (ou deux) domaine(s) pour le front. Le BFF **agrège** (« fiche produit = catalogue + stock + avis »). Si le service avis est down, le BFF peut renvoyer la fiche **sans** avis plutôt que tout casser.

Phrase d'entretien :

Je consomme un contrat d'API stable. Le découpage micro-services est un détail d'infra que le BFF me masque. Je gère les erreurs métier (409 stock), les timeouts, les états partiels.

Versioning et contrats

Les API évoluent (/v1/products, champs dépréciés). Le front **tolère** un champ en trop, et ne casse pas si un champ optionnel manque (? en TypeScript + fallback UI).

Les **feature flags** : on allume « nouveau tunnel » pour 10 % des users sans redéployer 12 services à la seconde près.

Micro-frontends (souvent à La Poste / BGPN)

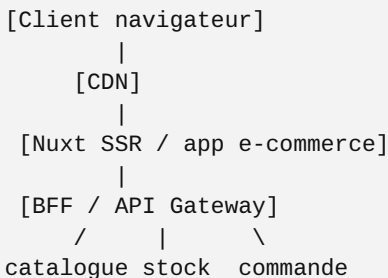
Un **micro-front** = un morceau d'UI **déployé séparément** (header groupe, suivi colis, tunnel) et assemblé dans une page.

Intérêt : une équipe « tracking » livre sans attendre la boutique.

Risque : **incohérence** du design system, double Vue, perf. D'où **Storybook + DS unique** (chapitre 12).

Tu peux citer : « je factorise les composants dans une lib DS ; les apps Nuxt les consomment ; on évite de recopier le bouton primaire ».

Ce que tu dessines au tableau (oral)



Plus Sentry (erreurs JS) et une CI GitLab. Ça suffit. N'invente pas 15 cases pour faire savant.

Chapitre 11 : le CSS qui ne pourrait pas en six mois.

11. SASS et BEM

CSS tout seul, le piège

En CSS global, `.title { font-size: 20px }` s'applique à **tous** les titres du site. Le titre du header casse le titre de la carte. À 40 pages, personne n'ose toucher.

Deux réponses dans l'offre : **SASS** (écrire le CSS plus confortablement) et **BEM** (nommer pour **isoler**).

SASS (ou SCSS)

SASS (Syntactically Awesome Style Sheets) est un **préprocesseur** : tu écris un fichier `.scss`, l'outil (Vite le fait) le transforme en CSS normal.

Ce que tu utilises vraiment :

Variables

```
$color-yellow: #ffc100;  
$color-ink: #1a1a1a;
```

Une couleur La Poste **un seul endroit**. Le DS ira plus loin avec des **tokens** (`--color-brand`).

Imbrication et &

```
.product-card {  
  &__title { font-weight: 700; }
```

```
&--out { opacity: 0.7; }  
}
```

Ça compile vers :

```
.product-card__title { font-weight: 700; }  
.product-card--out { opacity: 0.7; }
```

Le & = le sélecteur parent. C'est **fait pour BEM**.

Nesting trop profond

```
.page { .grid { .card { .title { } } } } /* à éviter */
```

Fragile, trop spécifique. Un block BEM plat est plus sain.

Dans Vue, on met souvent le SCSS en **scoped** : Vue ajoute un attribut unique pour que tes styles **ne fuent pas** vers les autres composants. BEM + scoped = ceinture et bretelles. L'offre demande BEM **quand même** (lisibilité, Storybook, DS).

BEM — Block Element Modifier

Convention de **noms de classes** :

```
block__element--modifier
```

- **Block** : le composant (product-card, app-shell).
- **Element** : une pièce **à l'intérieur**, liée au block (product-card__title). Séparateur : **deux** underscores.

- **Modifier** : un **état** ou une variante (product-card--out, product-card__cta--disabled). Séparateur : **deux** tirets.

HTML :

```
<article class="product-card product-card--out">
  <h2 class="product-card__title">Papier bulle</h2>
  <button class="product-card__cta" disabled>Ajouter au
  panier</button>
</article>
```

Règles d'or :

- pas de .product-card .title nu (le title n'est pas porté par le block) ;
- pas d'ID pour styler (#panier) ;
- un fichier / un composant $\approx$ un block ;
- on **n'enchaîne pas** __ deux fois (card__body__title : non, card__title : oui).

Pourquoi les recruteurs aiment : un junior **lit** le HTML et sait où est le CSS. Le DS Figma nomme souvent les mêmes blocks.

Responsive (rapide)

Le site doit marcher **mobile et desktop** (multi-navigateurs **et** multi-tailles). En CSS :

```
.catalog__grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(240px,
```

```
1fr));  
}
```

auto-fill + minmax : les cartes s’empilent toutes seules sur petit écran, sans media query obligatoire. Sinon @media (min-width: 768px) { ... }.

Chapitre 12 : d’où viennent les couleurs et les boutons « officiels ».

12. Design system

L'offre demande une **grande sensibilité au respect du Design System** et la connaissance **Frontify / Figma / Invision**.

C'est quoi un Design System (DS)

Un **design system**, ce n'est pas « une maquette jolie ». C'est le **langage commun** produit / design / dev :

- **tokens** : jaune La Poste, rayons 8 px, ombre, typo ;
- **composants** : bouton primaire, champ, carte, header ;
- **règles** : quand utiliser tel bouton, quels états (hover, disabled, error, loading) ;
- **accessibilité** : contrastes validés une fois, pas « au feeling ».

Sans DS, chaque équipe e-commerce invente un bouton. Le client a l'impression de changer de marque à chaque clic. Avec un DS, **Chronopost** et **boutique timbres** se sentent de la même maison.

Ta job : **implémenter** le DS, pas l'interpréter avec ton goût. Si la maquette dit 16 px, ce n'est pas 14 parce que « c'est plus aéré ».

Figma

Figma est l'outil où les designers **dessinent** les écrans (frames desktop / mobile), et souvent les composants du DS (variants).

Toi tu :

1. ouvres la frame « Catalogue » ;
2. notes les **écarts** : état vide ? erreur API ? rupture ? focus clavier ? (souvent **absents** — tu les **remonte** au designer, c'est la sensibilité UX de l'offre) ;
3. codes en tokens, pas en « pipette sur le PNG ».

Dev Mode / inspect : spacing, couleurs, typos. Tu ne codes pas au pixel près les yeux fermés.

Frontify

Frontify est une plateforme de **brand / DS documenté** : logos, couleurs officielles, règles d'usage, parfois les composants. Chez un grand groupe, c'est la **source de vérité marque**. Figma dessine le produit ; Frontify dit « ce jaune-là, ce logo-là ».

À l'oral : « je ne sors pas une couleur hors token Frontify / Figma ».

Invision

Invision (souvent **InVision**) est un **ancien** outil de prototype cliquable (avant que Figma fasse le proto). Tu peux encore voir des liens Invision dans des specs. Aujourd'hui le flux vivant = **Figma + Storybook**. Tu cites Invision pour montrer que tu lis une spec legacy, tu ne le proposes pas en 2026.

Storybook

Storybook est une **app à part** qui affiche tes composants **isolés**, sans le reste du site.

Pour un ProductCard tu crées des **stories** :

- InStock — bouton actif ;
- OutOfStock — disabled + texte rupture ;
- LongTitle — le titre ne casse pas la carte.

Intérêt :

- le designer **valide** le composant sans lancer tout Nuxt ;
- le junior **voit** les états ;
- les tests visuels / a11y addon tournent dessus ;
- la lib DS est **publiée** et versionnée (les micro-fronts la consomment).

Tu ne mets **pas** le métier panier dans le DS. Un bouton « Ajouter » du DS n'importe pas Pinia. La **page** catalogue câble le store.

Collaboration designer / back (offre « fonctionnelle »)

- Designer : « il manque l'état 0 résultat ».
- Back : « le 409 stock, c'est quel JSON ? ».
- Toi : tu ne bloques pas deux semaines : tu livres un **état par défaut** accessible, tu crées le ticket d'écart DS.

Chapitre 13 : prouver que ça ne cassera pas demain.

13. Tests, TDD et BDD

L'offre : « grande importance à la qualité du code : TDD, BDD » + **Jest** et **Cypress**.

Pourquoi tester un front e-commerce

Un tunnel de commande cassé = **perte de CA** et de confiance. Un refactor du panier sans tests = tu découvres la régression **en prod**.

Les tests sont un **filet**, pas une religion. On teste ce qui **peut casser silencieusement** : remise, stock, filtre, validation.

La pyramide (à dessiner)

```
      /\
     /E2E\      Cypress / Playwright – rares, chers,
vrais parcours
    /-----\
   /Composant\  Vue Test Utils – un bouton, une galerie
  /-----\
 /  Unitaire  \  fonctions, store Pinia – nombreux,
rapides
 /-----\
```

- **Beaucoup** de tests unitaires (Jest / Vitest).
- **Quelques** tests de composants.
- **Peu** de tests bout-en-bout (le parcours « ajouter → panier → payer »).

Si tu testes le `formatPrice` en Cypress, tu as perdu. Si tu ne testes le panier **que** à la main, tu as perdu aussi.

Jest vs Vitest

Jest est cité dans l'offre : c'est le runner historique (Vue 2 / Webpack).

Vitest est l'équivalent **Vite** / **Vue 3** : même style `describe / it / expect`. Dans ce kit on utilise Vitest. À l'oral : « même idée que Jest, branché sur Vite ».

```
it("privilégie le positif en cas d'égalité", () => {  
  expect(closestToZero([-5, 5])).toBe(5);  
});
```

- `describe` = un tiroir ;
- `it / test` = un cas ;
- `expect(...).toBe(...)` = l'assertion.

Vue Test Utils

On **monte** un composant sans navigateur visible (`jsdom` = faux DOM) :

```
const wrapper = mount(ImageGallery, { props: { links:  
  ["a.png", "b.png"] } });  
await wrapper.findAll(".remove")[0].trigger("click");  
expect(wrapper.findAll("img")).toHaveLength(1);
```

Ça vérifie le **comportement**, pas la beauté.

Cypress (et cousins Playwright)

Cypress ouvre un **vrai** navigateur, clique comme un humain :

```
Étant donné un catalogue  
Quand j'ajoute Colis S  
Alors le badge affiche 1
```

Lent, parfois flaky (attente réseau). On le réserve au **parcours critique**.

TDD — Test Driven Development

Ordre **Red** → **Green** → **Refactor** :

1. **Red** : tu écris un test qui **échoue** (la remise 10 % n'existe pas encore).
2. **Green** : tu écris **le minimum** pour qu'il passe.
3. **Refactor** : tu nettoies, les tests restent verts.

Intérêt : tu codes **le comportement demandé**, pas un château. En coding game, si les tests sont fournis, **lis-les d'abord** : ce **sont** l'énoncé.

BDD — Behavior Driven Development

On décrit le logiciel **en langage métier** (souvent Gherkin) :

```
Étant donné un produit en stock  
Quand le client l'ajoute au panier
```

Alors le badge affiche 1
Et le total TTC est à jour

Les **Given / When / Then** deviennent des steps Cypress. Product owner et dev **partagent** l'exemple. Ce n'est pas « plus de tests », c'est **le même test, nommé comme un récit**.

Quoi tester dans un panier (minimum senior)

- refus si stock 0 ;
- +1 si on ajoute deux fois le même SKU ;
- total et remise ;
- `setQuantity(0)` retire la ligne.

C'est exactement `tests/cart-store.test.ts`.

Chapitre 14 : un site qui marche aussi pour ceux qui ne voient pas le jaune.

14. Accessibilité WCAG

L'offre : « bonnes pratiques d'accessibilité », « WCAG 2.1 », « compatibilité multi-navigateurs ».

Pourquoi (pas seulement « c'est la loi »)

Un client peut :

- naviguer **au clavier** (pas de souris) ;
- utiliser un **lecteur d'écran** (JAWS, NVDA, VoiceOver) qui lit le HTML ;
- mal voir les contrastes ;
- ne pas distinguer le rouge du vert.

WCAG (Web Content Accessibility Guidelines) 2.1, niveau **AA**, est le palier visé par beaucoup de sites publics / grands groupes en France (souvent croisé avec le **RGAA**, le référentiel français dérivé de WCAG).

Accessibilité ≠ « pour les handicapés seulement ». Un bon contraste aide **dehors au soleil**. Un vrai <button> aide **tout le monde**.

Les 4 principes (POUR)

1. **Perceptible** — l'info n'est pas seulement une couleur ou une image sans texte.
2. **Utilisable** — clavier, assez de temps, pas de piège.

3. **Compréhensible** — labels clairs, erreurs explicites, langue fr.
4. **Robuste** — HTML valide, marche avec les aides techniques.

Checklist coding game (2 minutes, gros points)

1. **Un h1 par vue.** Les titres descendent (h2 dans les cartes, pas un autre h1).
2. **Boutons et liens natifs.** `<button type="button">`, `<a href>`.
Un `<div @click>` n'est pas dans la tabulation, n'a pas de rôle, casse le clavier.
3. **Chaque champ a un `<label for="id">`** lié au même id que l'input. Le placeholder **n'est pas** un label (il disparaît).
4. **Images :** alt utile (« Boîte colis format S ») ou alt="" si purement décoratif (sinon le lecteur lit le nom de fichier).
5. **Focus visible :** outline au clavier (:focus-visible). Ne pas outline: none sans remplacement.
6. **Contraste :** texte sombre sur fond clair / jaune. Gris pâle sur blanc = échec.
7. **Pas d'info par la couleur seule :** « Rupture de stock » **écrit**, pas seulement le bouton grisé.
8. **role="status"** pour « Chargement... » / « Aucun produit ». **role="alert"** pour une erreur de formulaire (annoncé tout de suite).
9. **Badge panier :** aria-live="polite" pour que le lecteur annonce « 2 » sans voler le focus.

10. **Formulaire** : erreurs **sous le champ** concerné, pas un toast illisible ; novalidate si tu gères toi-même, mais messages clairs.

Clavier : le test de 20 secondes

Tab, Tab, Tab. Est-ce que j'atteins recherche, selects, chaque « Ajouter », le lien panier ? Enter active le bouton ? Pas de piège (focus coincé) ?

ARIA, avec parcimonie

ARIA (Accessible Rich Internet Applications) = attributs `aria-*` pour **enrichir** quand le HTML natif ne suffit pas.

Règle : **un vrai bouton** > **un div avec `role="button"`**. ARIA ne répare pas un HTML pourri. Trop d'ARIA rend le site **plus** illisible.

Multi-navigateurs

« Compatible » = Chrome, Firefox, Safari, Edge, **et mobile**. Pas de fonction JS trop récente sans que Vite transpile. Pas de hover-only sur tactile (un filtre doit marcher au tap). Tester Safari pour le flex/grid et les 100vh.

Chapitre 15 : comment tout ça arrive sur un serveur sans copier les fichiers à la main.

15. GitLab, Docker, CI/CD

Git, en 2 minutes

Git enregistre l'histoire du code. **GitLab** (imposé par l'offre) héberge le dépôt, les **Merge Requests** (MR = proposition de fusion, l'équivalent des Pull Requests GitHub), et la **CI**.

Flux usuel :

1. branche feat/panier-remise ;
2. commits petits et clairs (feat : remise 10 % dès 50 €) ;
3. MR + revue (toi tu **revues les juniors**) ;
4. la CI doit être verte ;
5. merge dans la branche principale.

Tu ne pousses pas sur main en direct.

CI/CD

- **CI** (Continuous Integration) : à chaque push, une machine **installe, linte, teste, build**. Si rouge, on ne merge pas.
- **CD** (Continuous Delivery / Deployment) : ensuite on **déploie** (recette, préprod, prod), souvent via **Docker**.

Sans CI, « ça marche sur ma machine » n'est pas une preuve.

GitLab CI : le fichier `.gitlab-ci.yml`

GitLab lit ce YAML à la racine. Un **job** = une recette.

```
stages: [test, build]

test:
  stage: test
  image: node:22
  script:
    - npm ci
    - npm test
    - npm run lint

build:
  stage: build
  script:
    - npm ci
    - npm run build
  artifacts:
    paths: [dist/]
```

Vocabulaire :

- **image** : l'environnement (ici Node 22, comme un PC jetable).
- **npm ci** : install **reproductible** depuis `package-lock.json`. En CI, **préférer ci à install** (plus strict, plus stable).
- **script** : les commandes.
- **artifacts** : fichiers à garder (le build) pour le job suivant ou le déploiement.
- **cache** : `node_modules` pour aller plus vite (optionnel).

Tu n'écris pas 200 lignes le jour J. Tu dois **lire** un YAML et dire ce qu'il fait.

Docker

Docker met l'app dans une **boîte** (image) qui tourne pareil partout.

Analogie : au lieu d'installer Node « à la main » sur le serveur (version différente, oubli), tu livres une **image** : « voici le programme + exactement ce qu'il lui faut ».

Idée **multi-stage** (à citer) :

1. une étape **build** avec Node, compilateurs, `npm ci`, `npm run build` ;
2. une étape **finale** toute petite : `nginx` (fichiers statiques) **ou** Node pour Nuxt SSR (`node .output/server/index.mjs`).

La prod n'embarque pas les devDependencies ni le code source TS. Image plus petite = plus sûre, plus rapide.

Tu n'as pas à écrire un Dockerfile parfait au coding game. À l'oral : « build dans CI, image Docker, déploiement recette puis prod ».

Environnements

- **dev** : ta machine, `npm run dev`, mock API ;
- **recette / staging** : comme la prod, pour la QA et le métier ;
- **prod** : les clients.

Les URL d'API changent via les variables d'environnement (VITE_ / runtime Nuxt), pas en dur dans le code.

Chapitre 16 : le filet **avant** même les tests : le style du code.

16. ESLint et Prettier

L'offre les cite pour « qualité et formatage ».

Prettier — le formatage

Prettier réécrit le code **automatiquement** : quotes, point-virgules, retours à la ligne, longueur. Plus de guerre tabs vs espaces en revue.

Tu sauves, ça se formate. En CI, prettier --check refuse un fichier non formaté.

Ce n'est **pas** un détecteur de bugs. C'est un **accord visuel**.

ESLint — les règles métier / qualité

ESLint analyse le code et **signale** :

- une variable jamais utilisée ;
- un == au lieu de === ;
- v-for sans key (plugin Vue) ;
- un any interdit (avec TypeScript) ;
- un composant d'un seul mot (Gallery → ImageGallery).

En revue, tu ne commentes plus « mets une clé sur le v-for » : **la CI l'a déjà dit**.

Ensemble

Prettier formate. ESLint raisonne. Ils se marchent parfois sur les pieds : on désactive les règles ESLint de **style** au profit de Prettier (eslint-config-prettier).

Le jour J

Si le projet a `npm run lint`, lance-le. Si tu n'as pas le temps de configurer : **reste cohérent** (2 espaces, simples quotes, toujours des ; **ou** jamais). L'incohérence se voit plus que le choix.

Definition of Done d'une MR senior : lint vert, tests verts, a11y minimale, pas de `console.log` oublié.

Chapitre 17 : comment l'équipe s'organise autour de ces MR.

17. Agile / Scrum

L'offre : « expérience projets Agile/Scrum », plus l'encadrement, plus le relationnel.

L'idée

Agile = livrer **souvent** de la valeur, **ajuster** avec le métier, plutôt qu'un cahier des charges de 18 mois figé.

Scrum = un **cadre** d'équipe très répandu. Vocabulaire à utiliser correctement :

Mot	Sens
Sprint	Une période courte (souvent 2 semaines) avec un but
Backlog	La liste des travaux, priorisée
User story	Un besoin formulé côté user : « En tant que client, je peux filtrer les colis, afin de trouver l'offre S »
Critères d'acceptation	Quand c'est « fini » (les AC du coding game)
Daily	15 min : hier / aujourd'hui / blocage — pas un reporting de 40 min
Refinement	On découpe et on clarifie avant le sprint
Revue / démo	On montre le parcours au métier, pas une liste de MR
Rétro	Ce qu'on change dans notre façon de travailler
Definition of Done	Tests, a11y, DS, i18n, recette... le contrat qualité

SAFe

À l'échelle d'un groupe comme La Poste, on voit **SAFe** (Scaled Agile Framework) : plusieurs équipes Scrum **alignées** (PI Planning, trains). Tu n'as pas à réciter tout SAFe. Tu dis : « feature team front + back + PO + designer, sprints, démo du parcours client ».

Découper (compétence lead)

« Fiche produit + reco + avis + SEO + tracking » = **pas** un ticket. Tu proposes :

1. fiche + API + états erreur ;
2. SEO useSeoMeta ;
3. bloc avis ;
4. reco.

Sinon le sprint glisse, la démo est vide.

Encadrer un junior (c'est dans l'offre)

- **Pairing** 45 min sur le ticket réel, pas un kata hors sol.
- **Revue** : 1 point **bloquant**, 2 suggestions. Pas 40 nits de style (ESLint s'en charge).
- **DoD écrite** : « un composant DS a une story, un état disabled, un test, un nom BEM ».
- Tu **ne réécris pas** sa MR en silence : tu expliques, il corrige, il apprend.

Relationnel

« Esprit d'ouverture » dans l'offre = tu **challenge** une maquette incomplète **sans** humilier, tu **demandes** le contrat d'API au back **tôt**, tu dis « je ne sais pas encore, je reviens demain » plutôt que d'inventer un stock côté front.

Questions que **tu** poses en entretien (ça te classe senior) : DS déjà en Storybook ? Nuxt 2 ou 3 ? BFF unique ? Niveau RGAA contractuel ? Quel parcours est prioritaire (envoi vs boutique) ?

Chapitre 18 : ce que « performant » et « SEO » veulent dire en chiffres.

18. Performances et SEO

L'offre : « optimiser SEO et temps de chargement », « applications performantes ».

SEO, en une histoire

SEO = Search Engine Optimization. Google (et les autres) **lisent** ta page pour savoir si elle répond à « envoyer un colis pas cher ».

Ils lisent surtout :

- le **HTML** (titres, texte, liens) ;
- des balises (title, meta description, canonical, Open Graph) ;
- la **vitesse** et le **mobile** (oui, ça compte pour le ranking).

Si le HTML du catalogue n'existe qu'après 2 s de JavaScript (SPA nue), tu pars avec un handicap. D'où **Nuxt SSR** (chapitre 5).

Bon SEO fiche produit :

- un h1 unique = le nom de l'offre ;
- une description unique (pas le même lorem partout) ;
- une URL stable /produits/envoi-colis-s ;
- useSeoMeta ;
- pas de contenu important **uniquement** dans un canvas / image.

Le SEO n'est **pas** « bourrer des mots-clés ». C'est **une page claire, rapide, unique**.

Temps de chargement : les Core Web Vitals

Google a popularisé trois mesures **réelles** (pas un score Lighthouse vaniteux tout seul) :

Sigle	Nom	En français	Levier e-commerce
LCP	Largest Contentful Paint	Quand le gros élément (souvent l'image / le titre) s'affiche	SSR, image dimensionnée, pas 2 Mo de JS avant
INP	Interaction to Next Paint	Délai entre un clic et la réaction	moins de JS lourd, pas de gros travail sur le thread au clic
CLS	Cumulative Layout Shift	La page qui saute (le bouton se décale, tu cliques ailleurs)	largeur/hauteur des images, pas de pub qui pousse, skeletons de la même taille

Ancien sigle **FID** (First Input Delay) : remplacé par INP. Si le QCM dit FID, c'est la même famille.

Lighthouse (dans Chrome) simule. Le **RUM** (Real User Monitoring) mesure les vrais clients. À l'oral, tu vises les deux.

Gestes concrets (ceux que tu peux faire au coding game)

- computed pour filtrer, pas 12 fois la même boucle dans le template ;
- :key stable ;

- images : width / height ou aspect-ratio, loading="lazy" **sauf** l'image LCP (la première carte héro, elle, **pas** lazy) ;
- pas de fetch à chaque frappe (debounce) ;
- pas d'installer une lib de 80 ko pour un toggle ;
- v-if sur un gros bloc rarement vu.

Gestes Nuxt (oral)

- SSR du HTML above-the-fold ;
- routeRules / cache CDN ;
- ClientOnly pour le chat, le carousel lourd, la carte interactive ;
- payload : ne pas hydrater tout le site si une page est du contenu statique.

Multi-navigateurs et perf

Un polyfill énorme pour IE11 : en 2026, en général **non** (sauf contrainte écrite). Cible : navigateurs modernes + Safari iOS (souvent le plus strict).

Chapitre 19 : on ouvre **ton** mini-projet et on relie chaque fichier à ce cours.

19. Visite guidée du mini-projet

Tu as lu la théorie. Maintenant on ouvre les fichiers **dans l'ordre où le navigateur s'en sert**. Lance `npm run dev` à côté, clique, puis reviens au fichier.

La porte d'entrée

`index.html` → charge `src/main.ts`.

`main.ts` fait trois choses :

1. `createApp(App)` — démarre Vue ;
2. `app.use(createPinia())` — le cahier de caisse du panier est disponible **partout** ;
3. `app.use(router)` — `/`, `/panier`, `/commande` ;
4. `app.mount("#app")` — Vue prend possession du `<div id="app">`.

Sans Pinia **avant** les composants, `useCartStore()` explose.

Le chrome : `App.vue`

C'est le **layout** : `header + <RouterView />` (le trou où la page change).

Le badge lit count du store via `storeToRefs`. Clique « Ajouter » dans le catalogue : tu **ne** passes **pas** de props jusqu'au header. C'est **toute** la justification de Pinia (cours 06).

`aria-live="polite"` : le lecteur d'écran entend le nouveau nombre.

Classes `app-shell__header` : BEM (cours 11).

Les routes : `router.ts`

Trois pages, noms `catalog / cart / checkout`. `scrollBehavior` ramène en haut : détail UX d'une SPA (sinon tu restes au milieu de la page précédente).

En **Nuxt**, ce fichier n'existerait pas : `pages/index.vue`, `pages/panier.vue`, etc.

La page catalogue : `views/CatalogView.vue`

1. `useProducts()` — composable : `query`, `filters`, `filtered`, `fetchProducts`.
2. `onMounted(fetchProducts)` — **SPA**. En Nuxt ce serait `useFetch`.
3. `v-if loading / error / vide / liste` : les **quatre états** qu'un senior affiche toujours.
4. `ProductFilters` en `v-model:query` etc. — l'état vit dans le parent/composable, le filtre est bête.
5. `v-for + :key="product.id"` — jamais l'index.
6. `@add="addToCart" → cart.add(product)`.

Lis-le à voix haute : « je charge, je filtre en computed, je délègue l'UI, le store tient le panier ». C'est la réponse d'archi du coding game.

Le composable : `composables/useProducts.ts`

- `fetchProducts` simule une API (`setTimeout 250 ms`) pour que tu **voies** l'état `loading`. En vrai : `fetch / useFetch`.
- `filtered` est un **computed** qui enchaîne filtre texte + catégorie + tri. Changer query **recalcule**, pas besoin de `watch`.
- Le tri copie `[...list]` avant `sort` : `sort` mute le tableau ; on ne mute pas la source `products`.

La carte : `components/ProductCard.vue`

Composant **bête** : props `in`, emit `out`. Il ne connaît pas `Pinia`.
Avantage `Storybook` : tu montes une carte stock : `0` sans tout le site.

- `computed inStock` et `price` : le template reste lisible.
- `:disabled="!inStock"` + texte « Rupture de stock » = WCAG (couleur **et** texte).
- `aria-label` sur le bouton avec le **nom** du produit : utile si un jour le bouton n'est plus qu'une icône. Ici le texte visible existe déjà ; le label redonne un peu, ce n'est pas grave.

Les filtres : components/ProductFilters.vue

Chaque contrôle a un `<label for>`. Les `emit('update:query', ...)` sont le mécanisme des **v-model nommés**. Tu peux recoder ça au test les yeux fermés.

Le store : stores/cart.ts

Relis en parallèle le cours 06.

- `add : rupture → false ; même id → quantity++ ; sinon push.`
- `setQuantity : <= 0 → remove (l'input nombre du panier).`
- `totalCents` passe par `cartTotalCents` (remise 10 % dès 50 €)
: **la règle métier est testée sans Vue** dans `tests/algos.test.ts`. C'est du TDD sain : extraire la formule hors du composant.

Panier et checkout

`CartView.vue` : liste, `formatPriceEur`, lien vers commande.

`CheckoutView.vue` :

- `reactive` formulaire ;
- `errors` en **computed** (formules, pas un tableau que tu oublies de vider) ;
- affichage des erreurs seulement après `submitted` (on ne crie pas tant que le user n'a pas tenté) ;

- `novalidate` : on gère le message, tout en gardant `required` pour le sémantique / autofill ;
- succès → `cart.clear()` + retour catalogue.

C'est un **mini tunnel**. En vrai La Poste : adresse, point relais, paiement, 409 stock, idempotence. Ici on entraîne la **forme**.

Les types : `types/product.ts`

Le contrat. `CATEGORY_LABELS` : l'UI française d'un union TS. Une seule source pour le `<select>` et le libellé carte.

Les algos : `algos/codingGame.ts`

Fonctions **pures** (même entrée → même sortie, pas de DOM). Le coding game puzzle JS, c'est ça. Lis `closestToZero` jusqu'à pouvoir le réécrire sur papier.

Les exercices `TestDome : exercices/`

- `ImageGallery` : copie de prop + splice.
- `UserList` : push + reset des v-model.
- `LabeledInput` : slot + for/id.

Si le coding game n'est **pas** e-commerce, c'est **souvent** l'un de ces trois.

Les tests : dossier tests/

Lance `npm test`. Lis un test **avant** le code qu'il couvre. C'est exactement TDD inversé pour apprendre : le test **raconte** le comportement.

Ce qui n'est pas dans le mini-projet (volontairement)

Nuxt, GraphQL, Docker, Storybook, Cypress. Ce n'est pas un oubli : le coding game **tient** dans Vite + Vue + Pinia. Le **reste**, tu l'as dans les chapitres 5, 9, 12, 13, 15 pour le QCM et l'oral.

Quand un fichier du kit te paraît magique, reviens au chapitre du cours correspondant, puis relis le fichier. Deux passages valent mieux que dix tutos.

20. Glossaire

Lis-le en diagonal. Chaque entrée tient en **une phrase humaine**.

Mot	Signification
AA	Niveau WCAG visé par beaucoup de sites : plus strict que A, moins que AAA.
AC	Acceptance Criteria : les « c'est fini quand ».
API	Contrat d'échange entre programmes (front ↔ back).
ARIA	Attributs HTML pour aider les technologies d'assistance quand le HTML natif ne suffit pas.
BDD	Behavior Driven Development : tests racontés en langage métier (Given/When/Then).
BEM	Convention de noms CSS : Block__Element--Modifier.
BFF	Backend For Frontend : API adaptée au front, agrège les micro-services.
BGPN	Branche Grand Public et Numérique du Groupe La Poste.
CDN	Réseau de serveurs proches de l'utilisateur pour servir fichiers / HTML vite.
CI/CD	Intégration continue / déploiement continu.
CLS	Mesure de « la page qui saute » pendant le chargement.
Composable	Fonction Vue useX qui encapsule de la logique réactive réutilisable.
Composition API	Style Vue 3 (ref, computed, <script setup>).
CORS	Règle navigateur qui limite les appels HTTP vers un autre domaine.
CSR	Rendu uniquement dans le navigateur (SPA).

Mot	Signification
CSS	Langage de style.
Cypress	Outil de tests bout-en-bout dans un vrai navigateur.
Devtools	Outils du navigateur / Vue / Pinia pour inspecter.
Docker	Empaqueter l'app dans un conteneur reproductible.
DOM	Arbre d'objets de la page, manipulé par le navigateur et Vue.
DoD	Definition of Done : checklist qualité d'un ticket.
DS	Design System.
DSI	Direction des Systèmes d'Information.
DT	Direction Technique (ici eCommerce).
Emit	L'enfant Vue prévient le parent (« add », « update:query »).
ESLint	Analyseur de qualité de code.
ESM	Modules JavaScript natifs (import / export).
Fetch	Fonction navigateur pour appeler une HTTP API.
Figma	Outil de maquettes / DS.
Frontify	Plateforme de marque / documentation DS.
Git	Historique du code.
GitLab	Forge (dépôt, MR, CI) citée dans l'offre.
GraphQL	Langage de requête API : une URL, tu choisis les champs.
HTML	Structure de la page.
HTTP	Protocole des requêtes web (GET, POST, codes 200/404/409...).
Hydration	Nuxt/Vue rattache les événements au HTML déjà envoyé par le serveur.
INP	Mesure de réactivité au clic / frappe.
Invision	Ancien outil de prototype cliquable.

Mot	Signification
ISR / SWR	Régénérer / servir du HTML en cache puis le rafraîchir.
Jest	Runner de tests unitaires (historique).
JSON	Format texte d'objets, standard des API.
JSX	Syntaxe HTML-dans-JS (surtout React). Vue utilise surtout des templates.
LCP	Quand le plus gros contenu visible s'affiche.
MR	Merge Request GitLab.
Nuxt	Framework au-dessus de Vue : routes fichiers, SSR, conventions.
Options API	Ancien style Vue (data, methods).
Pinia	Store officiel Vue 3.
PO	Product Owner : priorise le métier.
Prettier	Formateur de code automatique.
Props	Données parent → enfant, en lecture seule.
Proxy	Mécanisme JS utilisé par Vue 3 pour la réactivité.
QA	Quality Assurance : recette / tests métier.
ref	Boîte réactive Vue. En script : <code>.value</code> .
REST	Style d'API : URL + verbes HTTP.
RGAA	Référentiel français d'accessibilité, dérivé WCAG.
Rollup	Bundler utilisé par Vite en production.
RUM	Mesure de perf chez les vrais utilisateurs.
SAFe	Cadre Agile à l'échelle de plusieurs équipes.
SASS / SCSS	Préprocesseur CSS (variables, nesting).
Scrum	Cadre Agile (sprints, daily, revue, rétro).
SEO	Être trouvable et compréhensible par les moteurs de recherche.

Mot	Signification
SPA	Single Page Application : un HTML, le JS change les vues.
SSR	Le serveur fabrique le HTML avant de l'envoyer.
SSG	HTML généré au build (statique).
SKU	Référence produit (id d'offre).
Storybook	Galerie de composants isolés.
TDD	Écrire le test avant le code (red/green/refactor).
TS	TypeScript.
useFetch	Composable Nuxt : GET SSR-friendly.
Vite	Bundler / dev server moderne (Vue 3, Nuxt 3).
Vitest	Jest-like pour Vite (ce kit).
Vue	Framework UI.
Vue Router	Routes côté client.
Vuex	Ancien store Vue 2.
WCAG	Recommandations d'accessibilité web (ici 2.1).
Webpack	Bundler historique (Nuxt 2, Vue CLI).
XSS	Attaque : injecter du JS via du HTML non filtré.

Tu as maintenant le vocabulaire de la fiche de poste. Retour au sommaire du cours, puis cheatsheets et QCM.

21. Questions fréquentes

« Vue et Nuxt, c'est deux frameworks ? »

Non. **Vue** peint les composants. **Nuxt** est la **maison** autour : pages-fichiers, serveur, SEO, auto-import. React / Next.js, c'est le même rapport.

« Pourquoi `.value` parfois et pas dans le HTML ? »

`ref` est une **boîte**. En JavaScript, tu ouvres la boîte (`count.value`). Dans le template, Vue l'ouvre pour toi (`{{ count }}`). Oublier `.value` en script = tu additionnes l'objet boîte, pas le nombre — résultat bizarre, pas d'erreur claire.

« Pourquoi mon badge panier ne bouge pas ? »

Tu as écrit `const { count } = useCartStore()`. Tu as sorti le nombre de la boîte. Utilise `storeToRefs`. C'est la question QCM.

« `computed` ou `watch` ? »

- Formule (total, liste filtrée, `isValid`) → **computed**.
- Action (appeler une API, écrire dans `localStorage`) → **watch**.

Si tu mets un `fetch` dans un `computed`, tu as mélangé les deux.

« Pourquoi pas tout dans App.vue ? »

Parce qu'au bout de 3 jours personne ne retrouve le filtre. Un composant = une responsabilité. Le store = la vérité partagée. Les pages = l'assemblage.

« On mute `items.value.push` : on n'avait pas dit immutable ? »

Vue 3 **autorise** de muter un ref tableau (push, splice) : la réactivité le voit. La règle « ne pas muter » concerne les **props** (la copie du parent). Le state Pinia, tu le mutes **dans le store**, pas depuis 4 composants au hasard.

« Centimes, c'est pédant ? »

$0.1 + 0.2 === 0.3$ est **faux** en JS (binaire des décimaux). Les banques et les boutiques comptent en **entiers**. Tu affiches ensuite avec `Intl`. Au coding game, ça évite un total 13.799999.

« SSR ça ralentit le serveur non ? »

Ça coûte du CPU serveur, oui. En échange le **client** voit le HTML tout de suite (LCP, SEO). On cache (CDN, SWR) pour ne pas recalculer 10 000 fois la home. Le tunnel loggé, lui, peut rester CSR.

« GraphQL va remplacer REST ? »

Non, pas dans cette mission. Les deux vivent. REST simple pour des ressources. GraphQL pour des fiches riches. Le BFF peut même **traduire**.

« Je dois tout connaître de Docker ? »

Non. Image = boîte. CI build l'image. Recette et prod lancent la même boîte. Multi-stage = on ne met pas les outils de compilation en prod. Point.

« WCAG 2.1 AA vs RGAA ? »

WCAG = standard W3C international. RGAA = **mode d'emploi français** pour les services publics / beaucoup de grands comptes. Les règles se recoupent (contraste, clavier, alternatives). Dis « AA / RGAA, labels, clavier, pas d'info par la couleur seule ».

« Storybook vs le site ? »

Storybook = laboratoire du bouton. Le site = le bouton **branché** au panier. On ne teste pas le paiement dans Storybook.

« Options API, je l'oublie ? »

Non. Du legacy Vue 2 / tests CodinGame CDN. Tu dois **lire** data / methods. Tu **écris** en `<script setup>` dès que le projet est Vue 3.

« Combien de temps pour être prêt ? »

Si tu codes déjà du Vue : **le cours 1 jour**, cheatsheets + QCM **une soirée**, mini-projet **une fois chrono**, oral **enregistré 15 min**. Si Vue est rouillé : fais le cours **dans l'ordre**, un chapitre = une pause, et recode `closestToZero` + le store panier **sans regarder**.

« Je peux utiliser l'IA le jour J ? »

Si le test est **proctorisé** (webcam, plein écran), non. Tu dois arriver **autonome**. Ce cours est fait pour ça.

[Retour au sommaire.](#)

Deuxième partie — Fiches jour J

Stratégie du coding game

Avant ce document : lis le **cours** (*cours/README.md*). Ici on parle **tactique de test**, pas des bases.

1. À quoi ça ressemble vraiment

Pour une mission **Vue.js / Nuxt e-commerce La Poste**, le « coding game » est en général **CodinGame for Work** (parfois CoderPad / TestDome / un take-home). Trois briques reviennent :

A. QCM (15–25 min)

Vue 3, JS/TS, parfois CSS, parfois Nuxt. Questions piège :

- ref vs reactive, destructuration qui casse la réactivité
- computed vs watch vs watchEffect
- v-model sur un composant enfant
- hydration SSR / useFetch vs onMounted + fetch
- key dans un v-for

B. Puzzle JS (10–25 min) — style CodinGame

Lecture stdin / fonction à implémenter. Les classiques :

- **Temperatures** : plus proche de 0, positif en cas d'égalité, [] → 0
- **Horse-Racing Duals** : écart min après tri

- **MIME Type** : map extension → type, casse, pas d'extension
- Calcul de panier / remise / groupBy

Règle d'or : **lire TOUS les cas de test avant d'écrire**. Le dernier cas casse presque toujours (tableau vide, égalité, 0 déjà présent).

C. Mini-app Vue (30–60 min) — c'est ça qui recrute

Un énoncé du type :

Afficher une liste (produits / colis / utilisateurs) depuis une API ou un tableau. Filtrer. Ajouter au panier / favoris. Gérer les états. Parfois un formulaire.

C'est **exactement** le mini-projet de ce kit (catalogue + panier + checkout).

2. Barème invisible (ce qu'ils notent vraiment)

Ils veulent voir	Ils pénalisent
MVP qui marche de bout en bout	Architecture over-engineered, rien n'est cliquable
Composition API + TS	Options API 2018, any partout
computed pour le dérivé	recopier la logique dans le template
états empty / loading / error	undefined is not iterable
labels, bouton disabled, focus	<div @click> sans clavier
BEM ou classes stables	CSS inline spaghetti
Pinia (ou un store propre)	props drill de 4 niveaux
tests s'ils sont fournis	ignorer les tests rouges

Pour un profil **5 ans + encadrement**, on attend aussi :

- nommage clair
- petits composants
- un commentaire **pourquoi** (pas **quoi**) si un choix n'est pas évident
- tu n'installes pas 12 libs pour un filtre

3. Plan de bataille le jour J (60 minutes chrono)

0–4 min — lire l'énoncé 2 fois, lister les AC Écris en commentaire en haut du fichier :

```
// AC: liste, filtre nom, add cart, qty, total, empty state
```

4–8 min — squelette

- types
- store ou ref
- composant liste + carte
- rien de joli encore

8–35 min — happy path Faire passer les tests / le scénario principal. Ne pas styler.

35–48 min — edge cases Stock 0, recherche vide, API en erreur, quantité max, remise.

48–58 min — polish scorable

- labels for/id
- :key stables (id, pas index)
- disabled + aria-label
- BEM sur 3 classes
- formatPrice en EUR

58–60 min — relire Console errors, tests, fautes de nommage.

Si tu n’as que 30 min : **happy path** + **empty** + **types**. Pas de checkout.

4. Modèle mental à coller sur l’écran

```

API / mock → composable (loading, error, data)
               ↓
             vue (filtre, tri) via computed
               ↓
          store Pinia (panier) source de vérité
               ↓
        composants bêtes (props + emit)
  
```

Interdit : fetcher dans 3 composants différents. Interdit : calculer le total dans le template avec 4 v-if.

5. Snippets à savoir taper les yeux fermés

Composant + v-model

```

<script setup lang="ts">
const model = defineModel<string>({ required: true });
</script>
  
```

```
<template>
  <input :value="model" @input="model = ($event.target as
HTMLInputElement).value" />
</template>
```

Ou Vue 3.4+ : `<input v-model="model" />` avec `defineModel`.

Fetch propre (SPA coding game)

```
const loading = ref(true);
const error = ref<string | null>(null);
const items = ref<Product[]>([]);

onMounted(async () => {
  try {
    const res = await fetch("/api/products");
    if (!res.ok) throw new Error("HTTP " + res.status);
    items.value = await res.json();
  } catch (e) {
    error.value = "Catalogue indisponible";
  } finally {
    loading.value = false;
  }
});
```

Fetch Nuxt (si le test est Nuxt)

```
const { data, pending, error, refresh } = await
useFetch<Product[]>("/api/products");
```

Ne jamais fetcher uniquement dans `onMounted` en Nuxt si tu veux du SSR / SEO.

Filtre + tri

```
const filtered = computed(() =>
  items.value
    .filter((p) =>
      p.name.toLowerCase().includes(q.value.toLowerCase()))
    .sort((a, b) => a.price - b.price),
);
```

6. Pièges qui font rater le test

1. **Destructurer un reactive / un store** sans `storeToRefs` → plus de réactivité.
2. **Muter une prop.** Toujours copier : `ref([...props.links])`.
3. **:key="index"** alors que la liste se réordonne / se filtre.
4. **Oublier type="button"** dans un formulaire → submit reload.
5. **Comparer des prix en float.** Travaille en **centimes** (1990 pas 19.90).
6. **JSON.parse sans try/catch.**
7. **Styliser pendant 20 min** avant que le filtre marche.
8. **Installer Vuex** sur un test Vue 3. C'est **Pinia**. Vuex = tu dois juste savoir le mapping (state/getters/mutations/actions).
9. **Ignorer l'accessibilité** alors que WCAG 2.1 est dans la fiche de poste. Un label et un button natif suffisent à marquer des points.

10. **Tricher avec une IA pendant un test proctorisé.** CodinGame / TestDome flaggent le copier-coller et le focus perdu. Tu dois arriver **autonome**.

7. Ce que tu t'entraînes à faire ici

- npm test : algos + galerie + user list + panier Pinia.
- npm run dev : coding game e-commerce complet.
- Relis les cheatsheets à voix haute : si tu hésites 5 secondes sur useAsyncData vs useFetch, tu n'es pas prêt pour le QCM.

8. Après le coding game

Le test technique n'est qu'un filtre. La mission se joue ensuite sur :

- design system (Frontify / Figma / Storybook)
- SSR / perf / SEO colis & boutique
- GitLab CI + Docker
- encadrement des confirmés

Prépare ça dans 06-entretien-oral.md. Ne te présente pas comme « un codeur Vue ». Présente-toi comme **un front e-commerce qui livre de l'expérience client mesurable**.

Cheatsheet Vue 3

Écris ces patterns de mémoire. C'est 80 % du coding game Vue.

Script setup + TypeScript

```
<script setup lang="ts">
import { computed, ref } from "vue";

const props = defineProps<{ title: string; count?: number }>();
const emit = defineEmits<{ increment: [value: number]; }>
();

const local = ref(props.count ?? 0);
const doubled = computed(() => local.value * 2);

function onClick() {
  local.value += 1;
  emit("increment", local.value);
}
</script>
```

- Props = descendantes, **immuables**.
- Events = remontent.
- v-model custom = modelValue + update:modelValue (ou defineModel()).

ref vs reactive

	ref	reactive
Primitive	oui (ref(0))	non (il boxe un objet)

	ref	reactive
Objet / array	ok (ref([]))	ok
Accès	.value en script	direct
Template	auto-unwrap	auto-unwrap
Destructuration	storeToRefs / toRefs	casse la réactivité

Règle simple le jour J : **ref partout**, reactive seulement pour un formulaire.

```
const form = reactive({ email: "", name: "" });
form.email = "a@b.c"; // OK
```

Directives à connaître

```
<p v-if="loading">...</p>
<p v-else-if="error">...</p>
<ul v-else>
  <li v-for="item in items" :key="item.id">{{ item.name }}
</li>
</ul>

<button @click="add(item)">Ajouter</button>
<input v-model.trim="q" />
<input v-model.number="qty" type="number" />
<p v-show="open">reste dans le DOM</p>
```

- v-if = montage / démontage.
- v-show = display. Pour un paragraphe toggle : v-show suffit (exercice TestDome).

computed / watch / watchEffect

```
const total = computed(() => qty.value * price.value); //  
dérivé pur, synchrone  
  
watch(q, (next, prev) => {  
  // appel API, debounce  
});  
  
watchEffect(() => {  
  console.log(cart.count); // s'exécute tout de suite + à  
chaque dep  
});
```

QCM piège : **ne pas** mettre de side-effect dans computed (fetch, push dans un array).

Cycle de vie (Composition API)

```
onBeforeMount(() => {});  
onMounted(() => {}); // DOM prêt – fetch SPA ici  
onBeforeUpdate(() => {});  
onUpdated(() => {});  
onBeforeUnmount(() => {});  
onUnmounted(() => {}); // clearTimeout,  
removeEventListener
```

Nuxt : préfère useAsyncData / callOnce au onMounted pour le SEO.

Composant enfant / parent

```
<!-- Parent -->  
<ProductCard :product="p" @add="cart.add" />
```

```
<!-- Enfant -->
<script setup lang="ts">
defineProps<{ product: Product }>();
defineEmits<{ add: [product: Product] }>();
</script>
```

Provide / inject seulement si 3+ niveaux (theme, locale), pas pour un panier.

Slots

```
<label :for="id"><slot /></label>
```

LabeledInput : le texte du label vient du slot, l'id de la prop. Relier for et id = point accessibilité gratuit.

Formulaires

```
<form @submit.prevent="onSubmit" novalidate>
  <label for="email">Email</label>
  <input id="email" v-model.trim="email" type="email"
autocomplete="email" />
  <p v-if="submitted && errors.email" role="alert">{{
errors.email }}</p>
  <button type="submit">Envoyer</button>
</form>
```

Toujours @submit.prevent. Toujours type="button" sur les boutons qui ne soumettent pas.

Réactivité : pièges QCM

```
const state = reactive({ list: [1] });
state.list.push(2);           // OK Vue 3
state.list = [...state.list, 3]; // OK

const n = ref(0);
n.value++;                    // obligatoire en script

const { count } = store;      // PERDU la réactivité
const { count } = storeToRefs(store); // OK
```

Vue 3 n'a plus le piège Vue 2 `Vue.set`. Mais **destructurer un ref objet interne** sans `toRefs` reste un piège.

Performance front (ils en parlent dans la fiche)

- `v-once` / `v-memo` rarement utiles en test.
- `:key` stable.
- `computed` pour filtres (pas recalculer dans le template 12 fois).
- `images : width/height, loading="lazy"` hors LCP.
- listes longues : pas de virtual scroll le jour J sauf si demandé.
- éviter les watchers qui refetch à chaque keypress → `debounce 300 ms`.

Vuex (ils écrivent « Vuex » / Vuex dans l'offre)

Tu dois pouvoir le recaser en 20 secondes :

```
state      → données
getters    → computed
```

```
mutations → sync, seules à modifier le state (Vuex 3)
actions    → async, commit des mutations
modules    → namespaced
```

En Vue 3 + Nuxt 3 : **Pinia remplace Vuex**. Mapping mental : state+getters = ref/computed du store, actions = fonctions. Pas de mutations.

Options API (si l'IDE du test est coincé en Vue 2)

```
export default {
  props: ["links"],
  data() {
    return { images: [...this.links] };
  },
  methods: {
    remove(i) {
      this.images.splice(i, 1);
    },
  },
};
```

Si le coding game charge Vue via un CDN sans build : Options API + pas d'import. Entraîne-toi aux deux.

Recette ImageGallery (exercice classique)

1. Copier la prop dans un ref local.
2. v-for + classe .image.
3. @click → splice(index, 1).
4. Ne pas muter props.links directement.

Cheatsheet Nuxt et Pinia

Cœur de la fiche de poste. Même si le coding game est une SPA Vite, **le QCM et l'oral** portent là-dessus.

Arborescence Nuxt 3 à réciter

```
app.vue
nuxt.config.ts
pages/                → file-based routing
  index.vue           → /
  produits/[id].vue   → /produits/:id
layouts/default.vue
components/           → auto-import
composables/useX.ts   → auto-import
stores/cart.ts        → Pinia
middleware/auth.ts
server/api/products.ts → /api/products
public/
```

Routes dynamiques : `pages/colis/[tracking].vue` → `const route = useRoute(); route.params.tracking.`

SSR / CSR / SSG / ISR

Mode	Quand	SEO	Exemple La Poste
SSR (ssr: true, défaut Nuxt)	HTML à chaque requête	excellent	fiche produit, home
SSG (nuxt generate)	HTML au build	excellent, moins frais	pages légales

Mode	Quand	SEO	Exemple La Poste
CSR (ssr: false)	SPA	mauvais	espace connecté, tunnel
ISR / SWR (route rules)	cache + revalidate	bon	catalogue qui bouge

Phrase à dire :

Sur un site e-commerce, le catalogue et les fiches sont en SSR pour le SEO et le LCP. Le tunnel de commande peut rester plus client-side, authentifié, sans indexer les étapes panier.

useFetch vs useAsyncData vs \$fetch

```
// Cas standard composant / page : GET, déduplicué, SSR-friendly
const { data, pending, error, refresh } = await
useFetch("/api/products", {
  query: { q: search },
  watch: [search],
});

// Quand tu transforms / appelles plusieurs sources
const { data } = await useAsyncData("product-42", () =>
$fetch("/api/products/42"));

// Hors setup (event handler, store) : PAS de SSR auto
async function addCoupon(code: string) {
  return await $fetch("/api/coupon", { method: "POST",
body: { code } });
}
```

Pièges :

- useFetch dans une fonction appelée au click = anti-pattern. Utiliser \$fetch.
- Double fetch hydration : **ne pas** refaire le fetch dans onMounted si useFetch a déjà tourné au SSR.
- Clé useAsyncData unique par ressource (product-`${id}`).

SEO Nuxt

```
useSeoMeta({
  title: product.value?.name ?? "Catalogue",
  description: product.value?.description,
  ogTitle: product.value?.name,
  ogImage: product.value?.image,
});

useHead({
  htmlAttrs: { lang: "fr" },
  link: [{ rel: "canonical", href: url }],
});
```

Core Web Vitals à citer : **LCP, INP, CLS**. Levier e-commerce :

- SSR du HTML au-dessus de la ligne de flottaison
- images dimensions fixes (pas de CLS)
- peu de JS au boot (Nuxt payloads, pas d'hydrate inutile → ClientOnly pour le chat, le carousel lourd)
- cache CDN / routeRules

```
// nuxt.config.ts
export default defineNuxtConfig({
  routeRules: {
    "/": { swr: 60 },
    "/produits/**": { swr: 300 },
    "/commande/**": { ssr: false },
  },
});
```

Pinia (à maîtriser)

```
export const useCartStore = defineStore("cart", () => {
  const items = ref<CartItem[]>([]);
  const count = computed(() => items.value.reduce((s, i) =>
s + i.quantity, 0));
  function add(product: Product) { /* ... */ }
  return { items, count, add };
});
```

Dans un composant :

```
const cart = useCartStore();
const { count, items } = storeToRefs(cart); // état
cart.add(product);                          // actions :
pas de storeToRefs
```

Persistence (souvent demandée à l'oral, rarement au test) : plugin `pinia-plugin-persistedstate` ou `localStorage` dans un `watch`, attention au SSR (`import.meta.client`).

REST vs GraphQL

Fiche de poste : les deux.

REST (le plus probable dans le test) :

```
await $fetch("/api/products", { query: { category: "colis"
} });
await $fetch("/api/cart/items", { method: "POST", body: {
sku, qty } });
```

Codes : 200 liste, 201 create, 400 validation, 401/403, 404, 409 stock, 422, 500. Afficher un message métier, pas le stack.

GraphQL :

```
query Products($q: String) {
  products(search: $q) {
    id
    name
    priceCents
    stock
  }
}
```

Avantage : pas de over-fetch sur une fiche produit riche.
Inconvénient : cache, erreurs partielles (errors[] + data), N+1 côté BFF. Citer **BFF / API Gateway** dans une archi micro-services
La Poste : le front ne parle pas à 12 micro-services, il parle à une API agrégée.

Micro-services / micro-fronts (oral)

Phrase utile :

*Le métier e-commerce est découpé (catalogue, panier, paiement, suivi colis). Le front Nuxt consomme des APIs via un BFF. Sur la DT, on voit aussi des **micro-fronts** (Storybook + design system) pour partager les composants (header, tracking, tunnel) entre sites BGPn, sans dupliquer le DS.*

Ne pas raconter Kubernetes. Parler **contrats d'API, versioning, feature flags, déploiements indépendants**.

Vite vs Webpack

	Vite	Webpack
Dev	ESM natif, ultra rapide	bundle complet
Prod	Rollup	Webpack
Nuxt 3	Vite par défaut	Nuxt 2 = Webpack

Savoir : vite.config.ts, alias @, env (import.meta.env.VITE_*), code splitting par route (Vue Router / Nuxt pages = auto).

GitLab CI — snippet à connaître

```
stages: [install, test, build]

cache:
  key: ${CI_COMMIT_REF_SLUG}
  paths: [node_modules/]

test:
  stage: test
```

```
image: node:22
script:
  - npm ci
  - npm run test
  - npm run lint

build:
  stage: build
  script:
    - npm ci
    - npm run build
  artifacts:
    paths: [.output/] # Nuxt
```

Docker : image Node pour le build, nginx ou node pour servir .output/public (SSG) ou node .output/server/index.mjs (SSR).

Storybook

Un composant = une story. Le DS La Poste (Frontify / Figma) arrive dans Storybook : states (default, hover, disabled, error), a11y addon, pas de logique métier dans l'UI kit.

```
// ProductCard.stories.ts (idée)
export const InStock = { args: { product: mockInStock } };
export const OutOfStock = { args: { product: mockEmpty } };
```

Cheatsheet TypeScript, SASS, BEM, a11y

TypeScript — le minimum scorable

```
interface Product {
  id: string;
  name: string;
  priceCents: number;
  stock: number;
}

type Category = "colis" | "emballage";

function add(product: Product, qty: number = 1): boolean {
  if (product.stock < qty) return false;
  return true;
}

const payload: Pick<Product, "id" | "name"> = { id: "x",
name: "y" };
```

Éviter any. Si l'API est inconnue : unknown + narrowing.

```
function isProduct(x: unknown): x is Product {
  return typeof x === "object" && x !== null && "id" in x
  && "priceCents" in x;
}
```

Génériques utiles : `groupBy<T, K>`, `uniqueBy<T, K>`.

SASS

```
$color-ink: #1a1a1a;
```

```
.product-card {
  &__title { font-weight: 700; }
  &--out { opacity: 0.7; }

  @media (min-width: 768px) {
    flex-direction: row;
  }
}
```

Variables, nesting, &, mixins si tu as 10 secondes. Pas de sélecteurs à 6 niveaux (ça casse le DS).

BEM — convention exigée par l'offre

```
block__element--modifier
```

```
<article class="product-card product-card--out">
  <h2 class="product-card__title">...</h2>
  <button class="product-card__cta">...</button>
</article>
```

Règles :

- Un block = un composant.
- Pas de `.product-card .title` nu.
- Modifiers pour état : `--disabled`, `--selected`, `--error`.
- Pas d'IDs CSS.
- Le DS Figma nomme souvent déjà les blocks : **respecte les tokens** (couleur, spacing), ne « fais pas joli à l'œil ».

Accessibilité WCAG 2.1 (niveau AA visé)

Checklist coding game (2 minutes de plus, gros points) :

1. Un seul h1 par vue.
2. Images : alt informatif, alt="" si décoratif.
3. Chaque input a un <label for="id">.
4. Boutons natifs, liens natifs. Pas de div @click.
5. :focus-visible { outline: 3px solid #ffc100; }.
6. Contraste : texte sombre sur jaune La Poste, pas gris pâle.
7. aria-live="polite" sur le compteur panier.
8. role="status" empty/loading, role="alert" erreur.
9. Formulaire : erreurs liées au champ, pas un toast illisible.
10. Clavier : Tab atteint tout, Enter active le CTA.

Piège e-commerce : **ne pas** indiquer le stock / le prix seulement par la couleur.

Frontify / Figma / Invision

À l'oral :

Je ne code pas au pixel près « de tête ». Je pars des frames Figma, des tokens du DS (Frontify), et je challenge l'écart (état vide manquant, erreur API, mobile). Invision = ancien flux de proto ; aujourd'hui Figma + Storybook.

Questions à poser au designer (ça te classe lead) :

- État rupture / 0 résultat / skeleton ?
- Error 500 ?
- Focus order du tunnel ?
- Contenu du alt et du titre SEO ?

Tests — TDD / BDD

Fiche de poste : « grande importance à la qualité : TDD, BDD ».

TDD : test rouge → code → vert → refactor. Sur un coding game, si Vitest/Jest est branché, **écris le test du filtre avant le CSS**.

BDD (Cypress / Gherkin) :

```
Étant donné un produit en stock
Quand j'ajoute au panier
Alors le badge affiche 1
Et le total TTC est à jour
```

Pyramide :

- **Jest/Vitest** : store, computed, formatPrice, composants isolés (@vue/test-utils).
- **Cypress / Playwright** : parcours critique (ajouter → panier → checkout).
- Pas de test E2E pour un getter de 3 lignes.

Exemple Vitest store : voir tests/cart-store.test.ts.

ESLint / Prettier

Le test peut lancer `npm run lint`. Active dès le début :

- pas de `var`
- `const` par défaut
- quotes / trailing comma via Prettier
- `vue/multi-word-component-names`

Si tu n'as pas le temps de configurer : **reste cohérent** (2 spaces, simple quotes, pas de `;` ou toujours `;`).

Git pendant un take-home

```
feat: catalogue filtrable  
feat: panier Pinia + remise 10%  
test: couverture store et ImageGallery
```

Pas un commit « wip » de 40 fichiers. Branches `feat/....` MR GitLab : description, screenshots, AC cochées.

QCM d'entraînement

Fais-les **sans notes**, 25 minutes chrono. Les réponses sont en bas.

1. Quelle est la différence entre `ref` et `reactive` pour un compteur `0` ?
2. Que se passe-t-il si on destructure `const { items } = useCartStore()` dans `<script setup>` ?
3. `computed` vs `watch` : lequel pour `total = qty * price` ?
4. Pourquoi `:key="index"` est dangereux sur une liste filtrable ?
5. `v-if` vs `v-show` ?
6. Comment implémenter `v-model` sur un composant enfant (Vue 3) ?
7. Où fetcher des produits dans une **SPA Vite** vs dans **Nuxt SSR** ?
8. `useFetch` dans un `@click` : bon ou mauvais ? Alternative ?
9. Hydration mismatch : cause fréquente ?
10. Pinia vs Vuex : que deviennent les mutations ?
11. Centimes vs euros en JS : pourquoi ?
12. Comment garder la réactivité d'un `props.links` que l'on va modifier (galerie) ?
13. `watchEffect` s'exécute-t-il immédiatement ?
14. Nuxt : fichier pour la route `/produits/abc` ?
15. middleware Nuxt : `navigateTo` se fait où ?
16. `ssr` : `false` sur `/commande/**` : intérêt ?

- !7. Core Web Vitals principaux ?
- !8. Comment limiter le CLS des cartes produit ?
- !9. BEM : `product-card__title--big` est-il valide ?
- !0. WCAG : un prix indiqué seulement en rouge est-il suffisant ?
- !1. REST : quel code si stock insuffisant au POST panier ?
- !2. GraphQL : que faire si data ET errors sont présents ?
- !3. Vite en dev bundle-t-il toute l'app comme Webpack ?
- !4. `import.meta.env.VITE_API_URL` vs `process.env` dans Vite ?
- !5. Jest vs Cypress : qui teste le store Pinia ?
- !6. TDD : ordre des 3 étapes ?
- !7. `debounce` : à quoi sert-il sur un champ recherche API ?
- !8. `nextTick` : à quoi ça sert ?
- !9. `shallowRef` vs `ref` : idée ?
- !0. `defineExpose` : quand ?
- !1. GitLab : différence `npm ci` vs `npm install` en CI ?
- !2. Docker multi-stage : pourquoi ?
- !3. Storybook : que documente-t-on pour un bouton DS ?
- !4. `ClientOnly` Nuxt : exemple légitime ?
- !5. Accessibilité d'un badge panier qui change : attribut ?
- !6. `aria-label` vs texte visible du bouton « Ajouter » ?
- !7. SASS `--disabled` génère quelle classe si le parent est `.btn` ?
- !8. TypeScript : `unknown` vs `any` ?

39. Pourquoi `JSON.parse` doit être dans un `try/catch` ?
40. Micro-service : le front Nuxt doit-il appeler 8 bases URL métier ?
-

Réponses

1. Un nombre va dans un `ref`. `reactive(0)` n'a pas de sens (objet requis). Accès : `n.value`.
2. Perte de réactivité. Utiliser `storeToRefs(store)` pour `items`, et appeler `store.add()` directement.
3. `computed.watch` = effet de bord (log, fetch).
4. L'index change quand on filtre/supprime → état DOM recyclé (input, animations, bugs).
5. `v-if` démonte ; `v-show` cache en CSS.
6. `Prop modelValue + emit update:modelValue`, ou `defineModel()`.
7. SPA : `onMounted + fetch`. Nuxt : `useFetch / useAsyncData` (SSR).
8. Mauvais. `$fetch / fetch` dans l'handler.
9. HTML SSR ≠ HTML client : `window`, `Date.now()`, `Math.random()`, `localStorage` au render, `v-if` différent.
40. Elles disparaissent. On mute les `ref` dans les actions Pinia.
11. `0.1 + 0.2 !== 0.3`. Entiers (centimes) puis `Intl.NumberFormat`.

- !2. Copier : `const images = ref([...props.links])`. Ne pas splice la prop.
- !3. Oui, puis à chaque dépendance.
- !4. `pages/produits/[id].vue`.
- !5. Dans `defineNuxtRouteMiddleware((to) => { if (!auth) return navigateTo('/login') })`.
- !6. Tunnel authentifié, pas de SEO, moins d'hydratation serveur, APIs privées.
- !7. LCP, INP (ex-FID), CLS.
- !8. Largeur/hauteur images, skeletons de même taille, pas de pubs qui poussent le DOM.
- !9. Oui : `block product-card`, `element title`, modifier `big`. Parfois on préfère `product-card__title` `product-card__title--big`.
- !10. Non. Couleur seule interdite (WCAG 1.4.1). Ajouter « Rupture », icône + texte.
- !11. Souvent **409 Conflict** ou **422**. Gérer le message « stock insuffisant ».
- !12. Afficher les données partielles + logger/afficher les erreurs champ par champ.
- !13. Non. Vite sert des modules ESM natifs, compile à la demande.
- !14. Vite expose `import.meta.env.VITE_*`. `process.env` n'existe pas au browser sans polyfill.
- !15. Jest/Vitest. Cypress = parcours navigateur.

- !6. Red → Green → Refactor.
- !7. Éviter un HTTP à chaque keypress. 250–400 ms.
- !8. Attendre que le DOM soit à jour après une mutation réactive.
- !9. shallowRef : seule .value est profonde... non : la **valeur racine** est réactive, le nested ne l'est pas. Utile gros objets immuables.
- !10. Pour exposer une méthode de composant au parent via ref template. Rare. Préférer emit.
- !11. npm ci = install reproductible depuis lockfile, plus strict, plus rapide en CI.
- !12. Image finale petite (nginx/node) sans compilateurs ni devDependencies.
- !13. Variants (primary/secondary), tailles, disabled, loading, icône, all, dark si DS.
- !14. Widget carte, chat, player, tout ce qui casse l'hydratation.
- !15. aria-live="polite" (et éventuellement aria-atomic="true").
- !16. Si le texte visible est déjà « Ajouter au panier », pas besoin d'aria-label. Si icône seule : aria-label obligatoire.
- !17. .btn--disabled.
- !18. any désactive le typage. unknown force un narrowing.
- !19. Payload API corrompu → exception, UI blanche sinon.
- !20. Non. BFF / API gateway. 1–2 bases URL max, tokens, CORS, agrégation.

Score : 32+/40 = prêt QCM. 24–31 = relire cheatsheets 02 et 03. < 24 = une journée de plus sur Nuxt/Pinia.

Entretien oral

Le coding game filtre. L'oral décide. Parle **métier e-commerce + expérience client**, pas seulement de syntaxe Vue.

Pitch de 60 secondes

Je suis développeur front Vue 3 / Nuxt, TypeScript, plutôt e-commerce et parcours client. Je construis des interfaces SSR performantes, branchées REST/GraphQL, collées à un design system. Je soigne l'accessibilité WCAG 2.1, les Core Web Vitals et la maintenabilité (Pinia, tests, Storybook, GitLab CI). Sur ce type de mission BGPn, je vise des tunnels (envoi colis, boutique, affranchissement) clairs, rapides, et tenables dans la durée — y compris en faisant monter les juniors.

Adapte avec **tes** vrais projets (un chiffre : LCP, conversion, couverture de tests, nb de composants DS).

« Présentez-vous sur un projet e-commerce »

Structure STAR :

- **Situation** : refonte front catalogue / tunnel.
- **Tâche** : Nuxt SSR, API panier, DS.
- **Action** : Pinia, useFetch, skeletons, a11y tunnel, tests Cypress checkout.

- **Résultat** : -X ms LCP, +Y % conversion, 0 régression tunnel.

Sans projet e-commerce : parle d'un **parcours critique** (formulaire, paiement, recherche).

Architecture que tu proposes pour la DT eCommerce

```
[Nuxt 3 SSR] → [BFF / API Gateway] → micro-services
                                     catalogue
                                     stock
                                     panier / commande
                                     paiement
                                     tracking colis
[Design System Storybook] ← Figma / Frontify
[GitLab CI] test → build Docker → recette → prod
```

Points à citer :

- **Découpage par parcours** (pages Nuxt) pas par techno.
- Store Pinia **par domaine** (cart, user, catalog), pas un god-store.
- Feature flags pour les campagnes.
- Observabilité front : Sentry + RUM (LCP réel).
- Feature teams Agile / SAFe (la BGPN fonctionne souvent comme ça).

SSR et SEO : la question piège

Pourquoi Nuxt et pas une SPA Vue ?

Réponse :

Les solutions e-commerce La Poste sont des portes d'entrée grand public (Google, campagnes, comparateurs). Sans SSR, le HTML initial est vide, le LCP et le SEO souffrent. Nuxt rend le catalogue côté serveur, hydrate ensuite l'interactivité (panier, filtres). Le tunnel loggé peut être moins SSR. On mesure LCP/INP/CLS, pas seulement un audit Lighthouse local.

Pinia vs Vuex

Vuex 4 existe encore, mais Pinia est l'officiel Vue 3 : DX, TS, pas de mutations, stores multiples, Devtools. Sur un legacy Vue 2 / Vuex, je mappe et je migre store par store.

Design system

Je ne « devine » pas les marges. Tokens Frontify/Figma → composants Storybook (BEM + SASS) → apps Nuxt. Si la maquette n'a pas l'état erreur, je le fais remonter au designer. C'est ça la sensibilité UX demandée, pas choisir une police.

Accessibilité

Cite 3 trucs concrets du tunnel :

1. Order Tab : quantité → retirer → CTA.

2. Erreurs de formulaire au `role="alert"` + focus sur le 1er champ.
3. Prix et rupture **textuels**, pas seulement couleur.
4. `prefers-reduced-motion` sur les carrousels promo.

Qualité / TDD / BDD

TDD sur le domaine (panier, remise, stock). BDD Cypress sur le parcours « ajouter un colis M → panier → adresse ». La CI GitLab casse la MR si rouge. Revue de code : a11y, naming, pas de logique dans le template.

Encadrement juniors (c'est dans l'offre)

Prépare 4 leviers :

- pairing 45 min sur un ticket réel (pas un kata hors sol)
- revue de MR pédagogique (1 point bloquant, 2 suggestions)
- dojo Storybook / a11y une fois par sprint
- définition of done écrite : tests, i18n, states, DS

Phrase :

Un junior qui livre un composant DS testé et accessible me fait plus gagner qu'un senior qui code tout seul dans son coin.

Agile

Daily, sprint 2 semaines, refinement avec métier + designer, démo du parcours (pas d'une MR technique), retros. Tu challenge les tickets trop gros (« fiche produit + reco + avis » = 3 tickets).

Questions à poser (esprit d'ouverture, excellent relationnel)

1. Le DS est-il déjà en Storybook ou encore éclaté Figma ?
2. Nuxt 2 ou 3 ? Pinia ou Vuex legacy ?
3. BFF unique ou appels directs catalogue / ATG / SAP Commerce / API colis ?
4. Niveau WCAG contractuel (AA) et outillage (axe, RGAA) ?
5. Parcours prioritaire : boutique, envoi, suivi ?
6. Comment sont mesurés satisfaction client / Web Vitals ?
7. Attente d'encadrement : % de code vs % de revue ?

Ça te positionne **prestation senior**, pas exécutant.

Questions pièges rapides

« **Vue 2 ou 3 ?** » Vue 3 + Composition API. Capable de maintenir du Vue 2 Options si socle legacy.

« **Webpack ou Vite ?** » Nuxt 3 / Vite. Webpack = Nuxt 2 / vieux CLI. Je sais configurer les deux.

« **Comment tu gères le stock concurrent ?** » Le front affiche le stock, mais la vérité est au POST commande (409). On resynchronise le panier, message clair, pas de double submit (bouton disabled + idempotency key si l'API le permet).

« **i18n ?** » vue-i18n / @nuxtjs/i18n, clés dans le DS, dates/prix fr-FR. Ne jamais concaténer des phrases.

« **Sécurité front ?** » XSS (v-html évité), cookies HttpOnly pour la session, CORS géré au BFF, pas de secret dans VITE_*.

Checklist jour J

La veille

- ☐ Dors. Pas de nouvelle lib à 23 h.
- ☐ Relis **une fois** : 02-cheatsheet-vue3.md + Pinia/useFetch.
- ☐ Refais de mémoire `closestToZero` et un `computed` filtre.
- ☐ `npm test` au vert sur ce kit.
- ☐ Vérifie matériel : casque, webcam si proctor, Chrome à jour, IDE du test (souvent VS Code embedded).
- ☐ Espace calme, notifications off, téléphone loin **si** test surveillé.

30 minutes avant

- ☐ Eau, full screen, second écran **seulement si autorisé**.
- ☐ Papier : AC, types Product, schéma store.
- ☐ Raccourcis : dupliquer ligne, multi-curseur, formatage.
- ☐ Tu n'ouvres pas ChatGPT si le règlement l'interdit.

Pendant le QCM

- ☐ 30 s max par question, flag, reviens à la fin.
- ☐ Sur un doute Vue 2 vs 3, choisis **Vue 3** sauf si l'énoncé montre `this.$store`.

Pendant le puzzle JS

- ☐ Lis l'exemple **et** les contraintes ($N = 0$, égalité, casse).
- ☐ Écris d'abord la signature + `if (empty) return 0`.
- ☐ 1 passe $O(n)$ si possible. Pas de $O(n^2)$ gratuit.
- ☐ `console.error` de debug, **pas** sur `stdout` si le puzzle lit `stdout`.

Pendant le mini-app Vue

Ordre non négociable :

1. Types + fake data
2. Affichage liste
3. Filtre
4. Panier / action
5. Empty / loading / disabled
6. Labels + :key
7. CSS BEM léger
8. S'il reste 10 min : tests ou formulaire

Phrase si tu n'as pas fini (commentaire README du take-home) :

Fait : catalogue, filtre, panier, rupture de stock, a11y labels. Non fait : persistance localStorage, tests E2E Cypress, page produit. Prochaine étape : useFetch Nuxt + SEO useSeoMeta.

Ça montre un senior, pas quelqu'un noyé.

Après l'envoi

Note 5 minutes : ce qui t'a ralenti. Tu auras un oral de debrief.

Commandes de révisions express

```
cd prep-mission-laposte  
npm test  
npm run dev
```

Parcours manuel à faire **une fois** le matin du test :

1. Chercher « colis »
2. Trier prix croissant
3. Ajouter Colis S deux fois
4. Voir le badge
5. Modifier la quantité
6. Aller au checkout, valider avec un email faux → erreur
7. Email bon → retour catalogue, panier vide

Entraînement chrono

Fais ça **sans ouvrir les solutions** (src/ est la solution : pour t'entraîner, recode dans un fichier vierge).

Session A — 20 min — algos

Dans un fichier vide, implémente et teste à la main :

1. `closestToZero([-5, 5, 2])` → 2 ; `[-5, 5]` → 5 ; `[]` → 0
2. `minHorseGap([3, 10, 8])` → 2
3. `cartTotalCents([ { unitPriceCents: 3000, quantity: 2 } ])` → 5400

Puis `npm test` et compare `src/algos/codingGame.ts`.

Session B — 45 min — Vue e-commerce

Consigne (l'énoncé type) :

Tu reçois une liste de produits e-commerce. Affiche-les en cartes. L'utilisateur peut rechercher par nom, filtrer par catégorie, trier par prix. Il peut ajouter au panier si le stock le permet. Le header affiche le nombre d'articles. Une page panier permet de changer les quantités. Remise 10 % dès 50 €. Accessibilité minimale (labels, boutons natifs). BEM.

Contraintes :

- Vue 3 <script setup> + TypeScript
- Pinia
- Pas de lib UI
- États : loading, vide, rupture

Compare ensuite avec `src/views/` et `src/stores/cart.ts`.

Session C — 20 min — exercices « TestDome »

Recode sans copier :

- galerie d'images + bouton remove
- table d'utilisateurs + reset des inputs
- LabeledInput label lié à l'input

Les tests sont dans `tests/components.test.ts`.

Session D — 25 min — QCM

`docs/05-qcm.md` sans scroller les réponses.

Session E — 15 min — oral

Enregistre-toi au téléphone sur :

1. Pitch 60 s
2. Pourquoi Nuxt pour La Poste ?
3. Comment tu fais monter un junior ?

Si tu pites, relis `docs/06-entretien-oral.md` et recommence.

Snippets à recoller

1. Page liste + filtre (Vue 3)

```
<script setup lang="ts">
import { computed, onMounted, ref } from "vue";

interface Item {
  id: string;
  name: string;
  price: number;
}

const q = ref("");
const loading = ref(true);
const error = ref<string | null>(null);
const items = ref<Item[]>([]);

onMounted(async () => {
  try {
    const res = await fetch("/api/items");
    if (!res.ok) throw new Error("http");
    items.value = await res.json();
  } catch {
    error.value = "Impossible de charger les données.";
  } finally {
    loading.value = false;
  }
});

const filtered = computed(() => {
  const needle = q.value.trim().toLowerCase();
  return items.value.filter((item) =>
    item.name.toLowerCase().includes(needle));
});
</script>
```

```

<template>
  <label for="q">Recherche</label>
  <input id="q" v-model.trim="q" type="search" />
  <p v-if="loading" role="status">Chargement...</p>
  <p v-else-if="error" role="alert">{{ error }}</p>
  <p v-else-if="filtered.length === 0" role="status">Aucun
résultat.</p>
  <ul v-else>
    <li v-for="item in filtered" :key="item.id">{{
item.name }}</li>
  </ul>
</template>

```

2. Store Pinia panier

```

export const useCartStore = defineStore("cart", () => {
  const lines = ref<{ id: string; qty: number; price:
number }[]>([]);
  const count = computed(() => lines.value.reduce((s, l) =>
s + l.qty, 0));
  const total = computed(() => lines.value.reduce((s, l) =>
s + l.qty * l.price, 0));
  function add(id: string, price: number) {
    const line = lines.value.find((l) => l.id === id);
    if (line) line.qty += 1;
    else lines.value.push({ id, qty: 1, price });
  }
  return { lines, count, total, add };
});

```

3. Nuxt page produit

```

<script setup lang="ts">
const route = useRoute();

```

```
const { data: product, error, pending } = await
useFetch(`/api/products/${route.params.id}`);
useSeoMeta({ title: () => product.value?.name ?? "Produit"
});
</script>
```

4. closestToZero

```
function closestToZero(xs: number[]): number {
  if (xs.length === 0) return 0;
  return xs.reduce((best, n) => {
    if (Math.abs(n) < Math.abs(best)) return n;
    if (Math.abs(n) === Math.abs(best)) return
Math.max(best, n);
    return best;
  });
}
```

5. GitLab job minimal

```
test:
  image: node:22
  script:
    - npm ci
    - npm test
    - npm run build
```

Mapping fiche de poste

Exigence de l'offre	Preuve coding game	Preuve oral
Vue.js 3.x 5 ans	Composition API, props/emit, computed, liste	Diff Vue 2/3, migration
Nuxt SSR routes dynamiques	QCM pages/[id].vue, useFetch	SEO catalogue colis
Webpack ou Vite	Vite alias, env	Nuxt 3 = Vite, Nuxt 2 = Webpack
Storybook	optionnel	stories d'états DS
GitLab CI + Docker	snippet YAML	pipeline MR
ESLint Prettier	code propre	DoD
Jest + Cypress	Vitest store/composants ici	pyramide + parcours tunnel
REST / GraphQL	fetch + erreurs HTTP	BFF, over-fetch, 409 stock
Vuex / Pinia	store panier	mapping mutations → actions
Micro-services	—	BFF, pas 8 bases URL
TypeScript	interfaces Product	unknown vs any
SASS + BEM	cartes produit	tokens DS
Frontify/Figma/Invision	—	« je code le DS, je ne l'invente pas »
TDD/BDD	tests du kit	Gherkin checkout
Encadrer juniors	—	revue, pairing, DoD
UX/UI	empty/error/focus	challenge maquette

Exigence de l'offre	Preuve coding game	Preuve oral
WCAG 2.1	labels, roles	RGAA AA, pas couleur seule
Agile/Scrum	—	tickets trop gros, démo parcours
Perf / SEO	computed filtre, pas de fetch spam	LCP INP CLS, routeRules

Si tu ne peux remplir que la colonne « coding game », tu passes le test et tu rates la mission. Travaille les deux.